

# **Introduction to Vulkan**

# About Vulkan

---

## **Vulkan**

- A new industrial standard for high-performance graphics
- **Before Vulkan**
  - OpenGL: Not optimized for parallel multi-GPU computing
  - DirectX: Since DirectX 12.0 it became similar to Vulkan
  - Metal: Apple version of Vulkan (although it comes out earlier)
  - Mantle: AMD version of Vulkan (although it comes out earlier)

# About Vulkan

---

## Why Vulkan?

- GPU optimized: 10~100 times faster than OpenGL /DirectX 11 or before
- Graphics driver is now getting slimmer
- Why not DirectX12 or Metal?
  - Vulkan is cross-platform

## Why not Vulkan?

- Need to control almost every detail of GPU
- Framebuffer creation, Memory managements for parallel processing, etc

# About Vulkan

---

- Vulkan is for core programmers
  - Low level, high performance tool
- If you are interested only in the graphics application, you may want to use high level tools such as;
  - OpenSceneGraph
  - Ogre3D
  - Unity3D
  - Unreal 5
- If you are interested in general graphics application, you may want to use relatively high-level API such as;
  - OpenGL / WebGL / OpenGL ES

# **What we will be learning is...**

---

**01. Introduction to Vulkan + Fundamentals Setup**

**02. Managing Resources in Vulkan (Buffers)**

**03. Textures and Other Resources**

**04. Parallel Programming with Vulkan**

**05. Ray Tracing with Vulkan**

# What we will be learning is...

---

## 01. Introduction to Vulkan & Fundamentals Setup

- Setup
- Graphics pipeline
- Drawing

## 02. Managing Resources in Vulkan (Buffers)

- Vertex Buffers
- Uniform Buffers

## 03. Textures and Other Resources

- Texture Mapping
- Depth Buffering

## 04. Parallel Programming with Vulkan

## 05. Ray Tracing with Vulkan

- Types of Shaders
- Adding Shader
- Shader Binding Table

# **01. Introduction to Vulkan & Fundamentals Setup**

# Objectives of this Lecture

---

- **Getting to Know Vulkan Architecture**
  - Follow the key steps from instance creation to rendering
- **Drawing a Simple Triangle with Vulkan**
  - Practice assignment 'ex01'





# Practice assignment

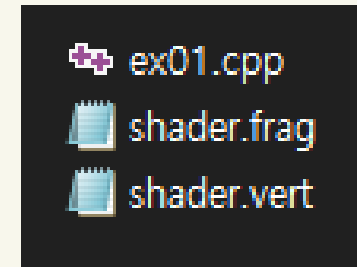
---

- **Setup Vulkan**

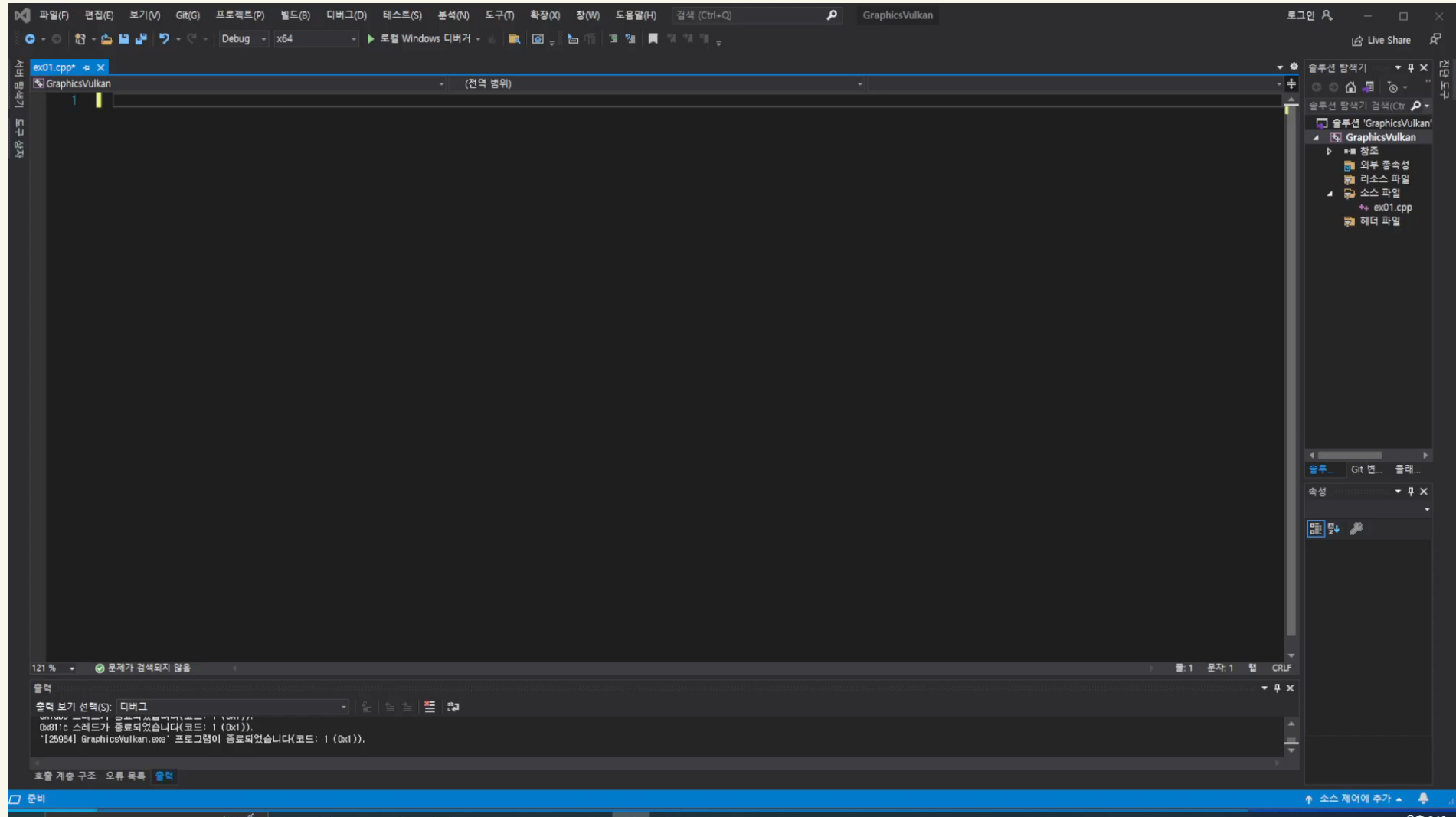
- Vulkan, glfw, glm download
- Visual Studio project settings

- **Drawing a Triangle**

- Check file name before compiling shader



# Let's Set Up



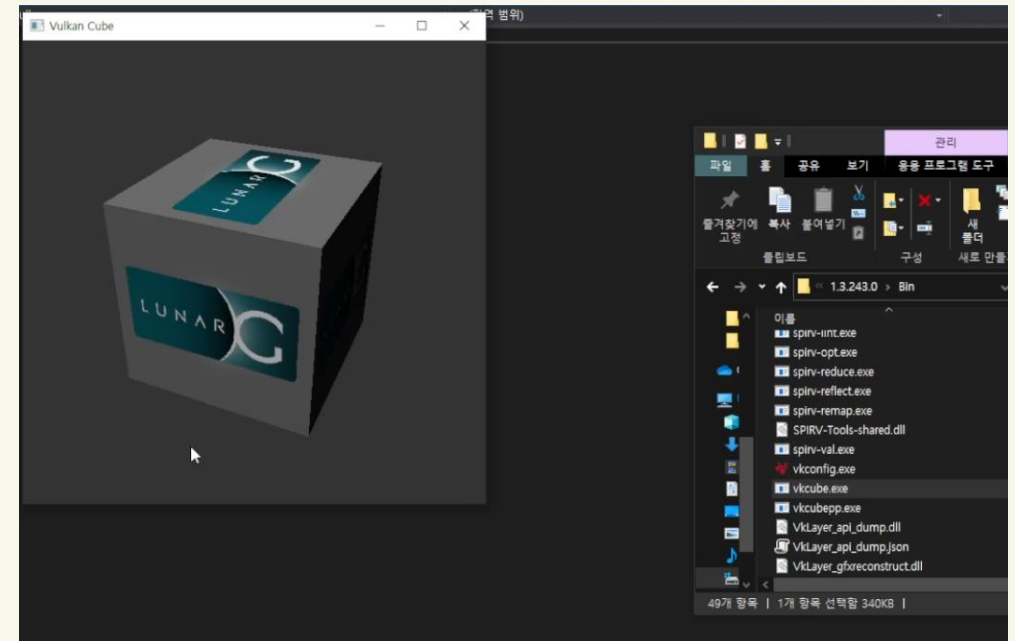
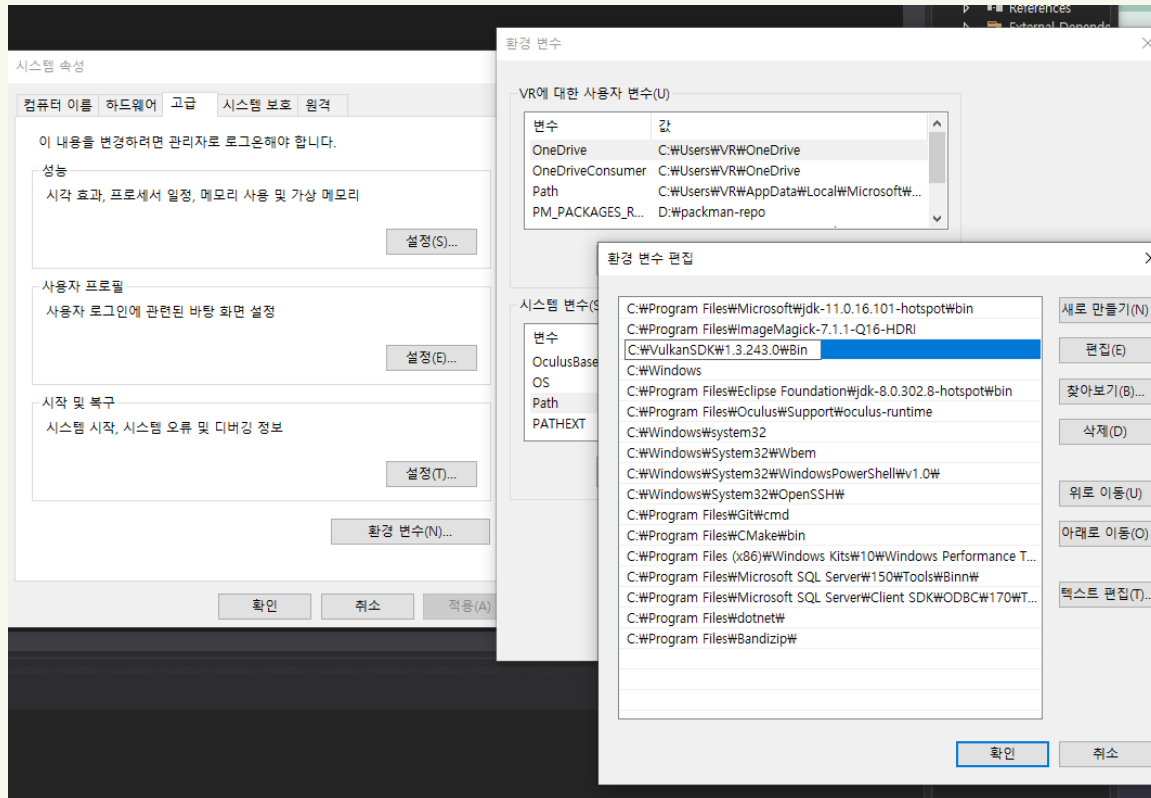
# Let's Set Up

---

- **Vulkan SDK**
  - <https://vulkan.lunarg.com/>
- **GLFW**
  - <https://www.glfw.org/download.html>
- **GLM**
  - <https://github.com/g-truc/glm/releases>

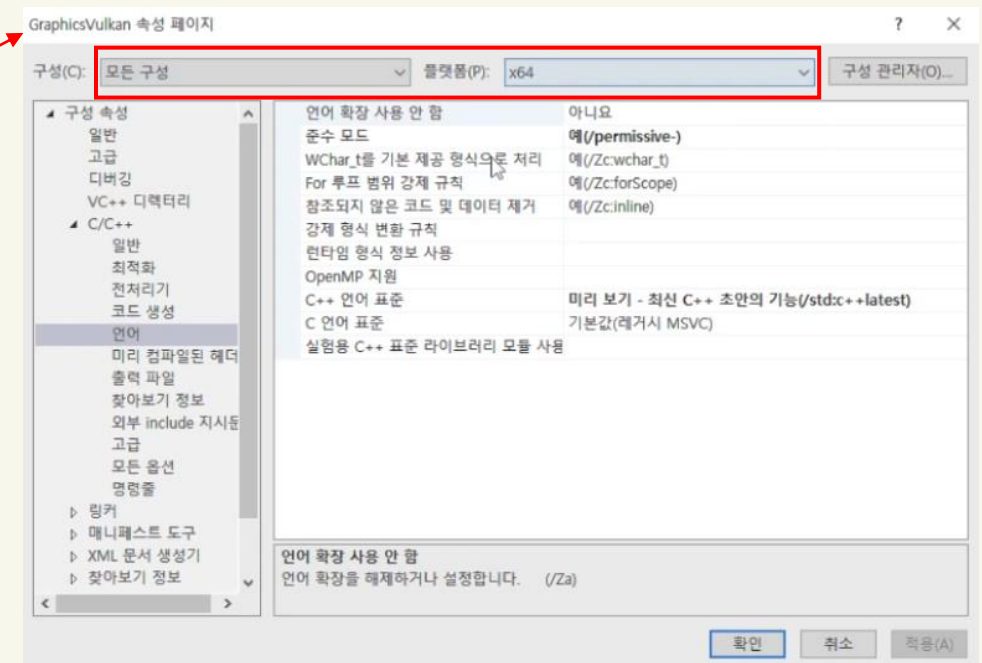
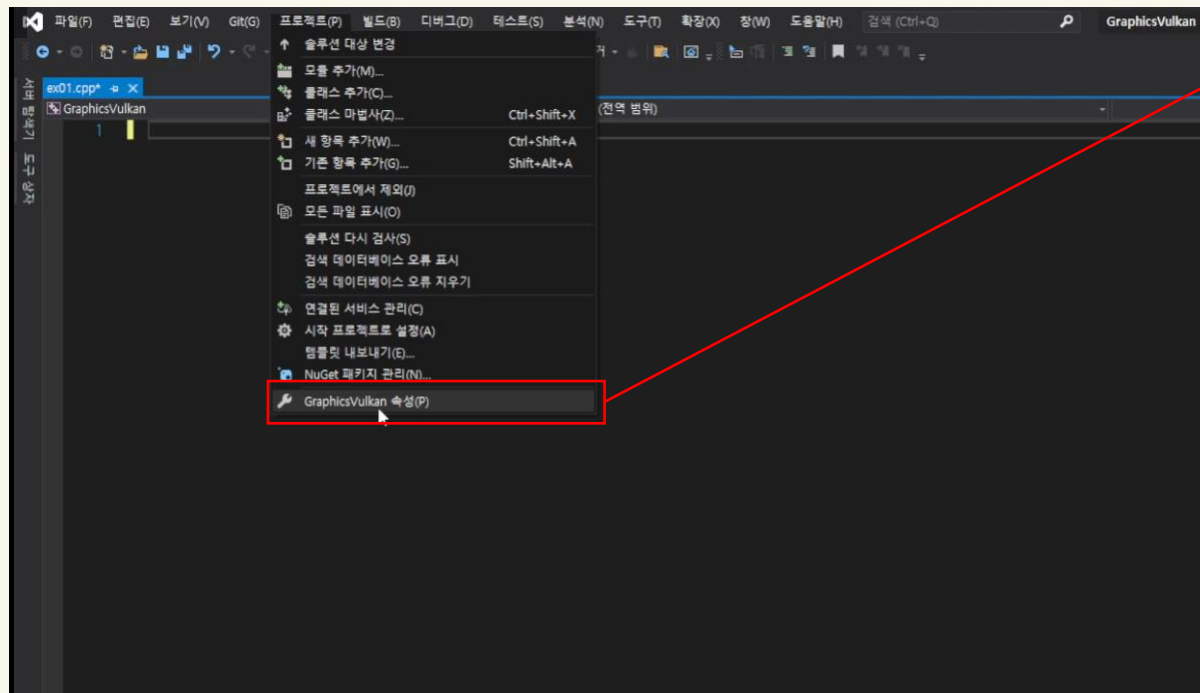
# Let's Set Up

- Setting environment variables
- Run this file : `C:\VulkanSDK\1.3.243.0\Bin\vkcube.exe`
  - Check if Vulkan is available

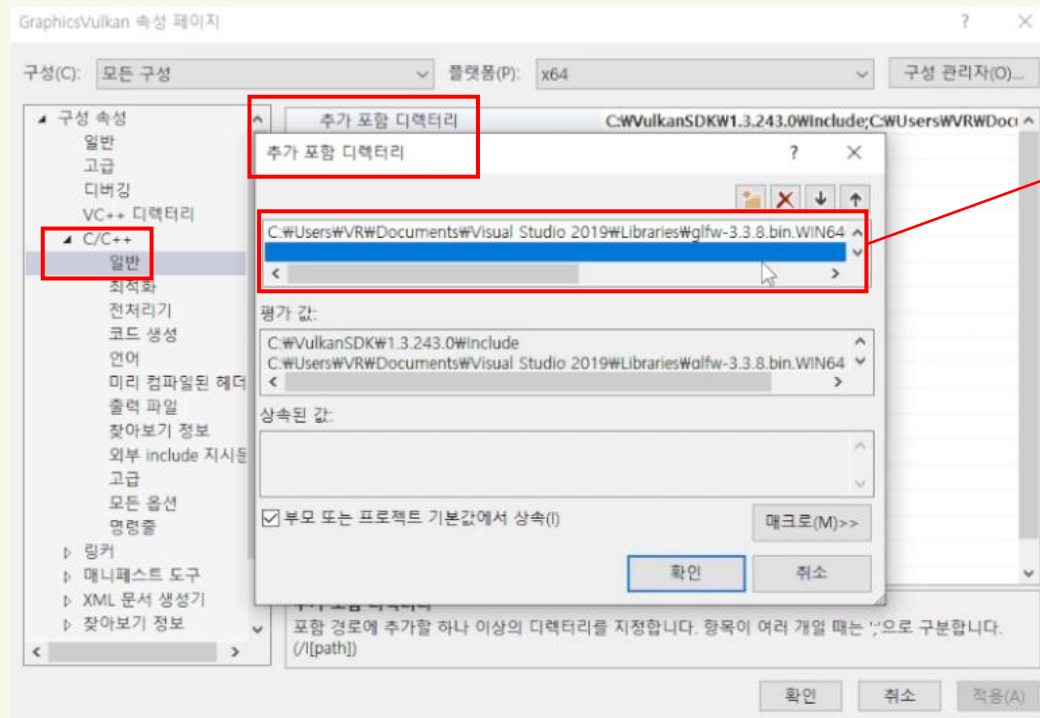


# Let's Set Up

- Properties page
- Setting Configuration

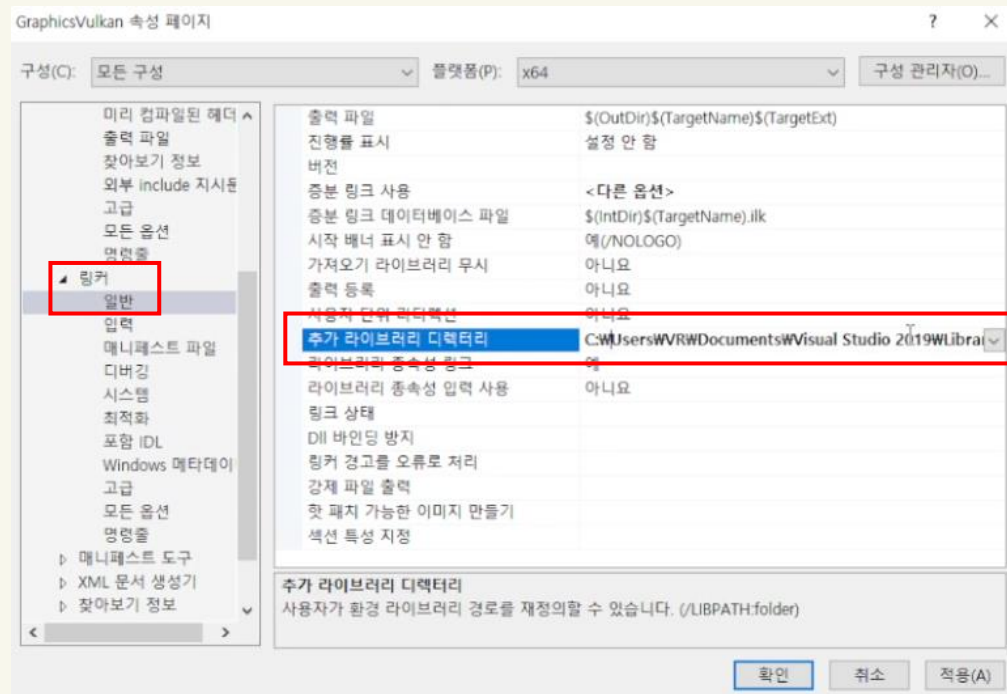


# Let's Set Up



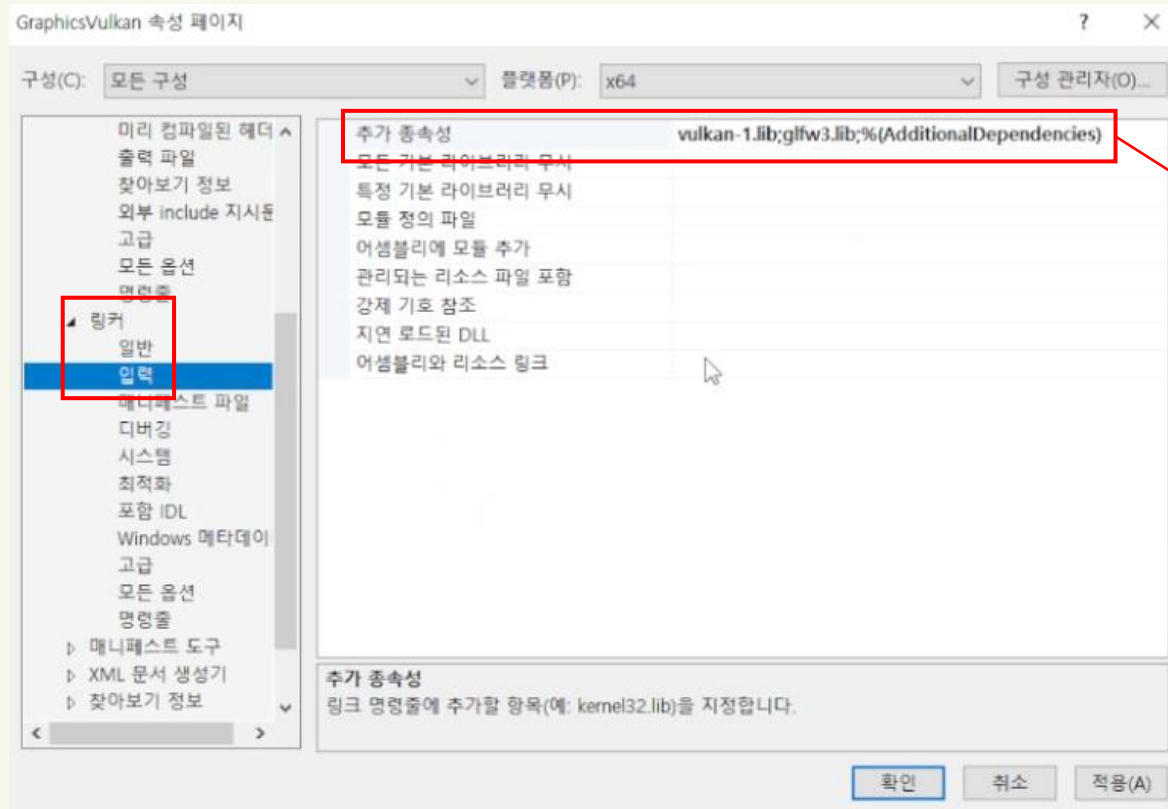
- Add path to 'Additional Include Directories'
1. C:\VulkanSDK\1.3.243.0\Include
  2. C:\YOUR PATH\glfw-3.3.8.bin.WIN64\include
  3. C:\YOUR PATH\glm

# Let's Set Up



- Add path to 'Additional Library Directories'
- C:\VulkanSDK\1.3.243.0\Lib
- C:\YOUR PATH\glfw-3.3.8.bin.WIN64\lib-vc2019

# Let's Set Up



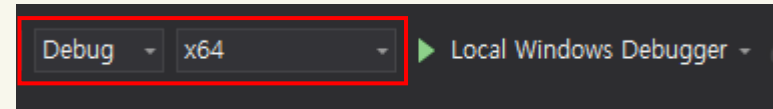
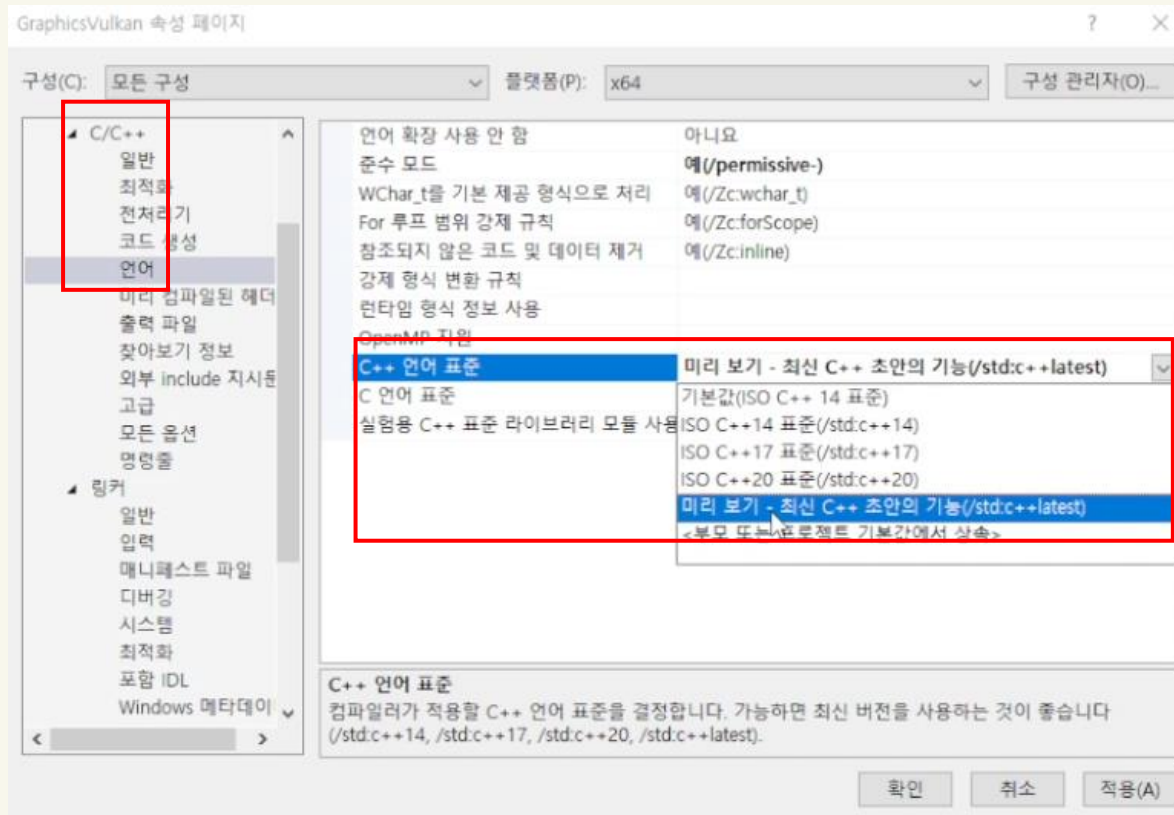
• Add to 'Additional Dependencies'

- vulkan-1.lib
- glfw3.lib



# Let's Set Up

- Setting C++ Language Standard



- Check shader path and file name

```
void createGraphicsPipeline() {  
    auto vertShaderCode = readFile("shaders/shader.vert");  
    auto fragShaderCode = readFile("shaders/shader.frag");  
}
```

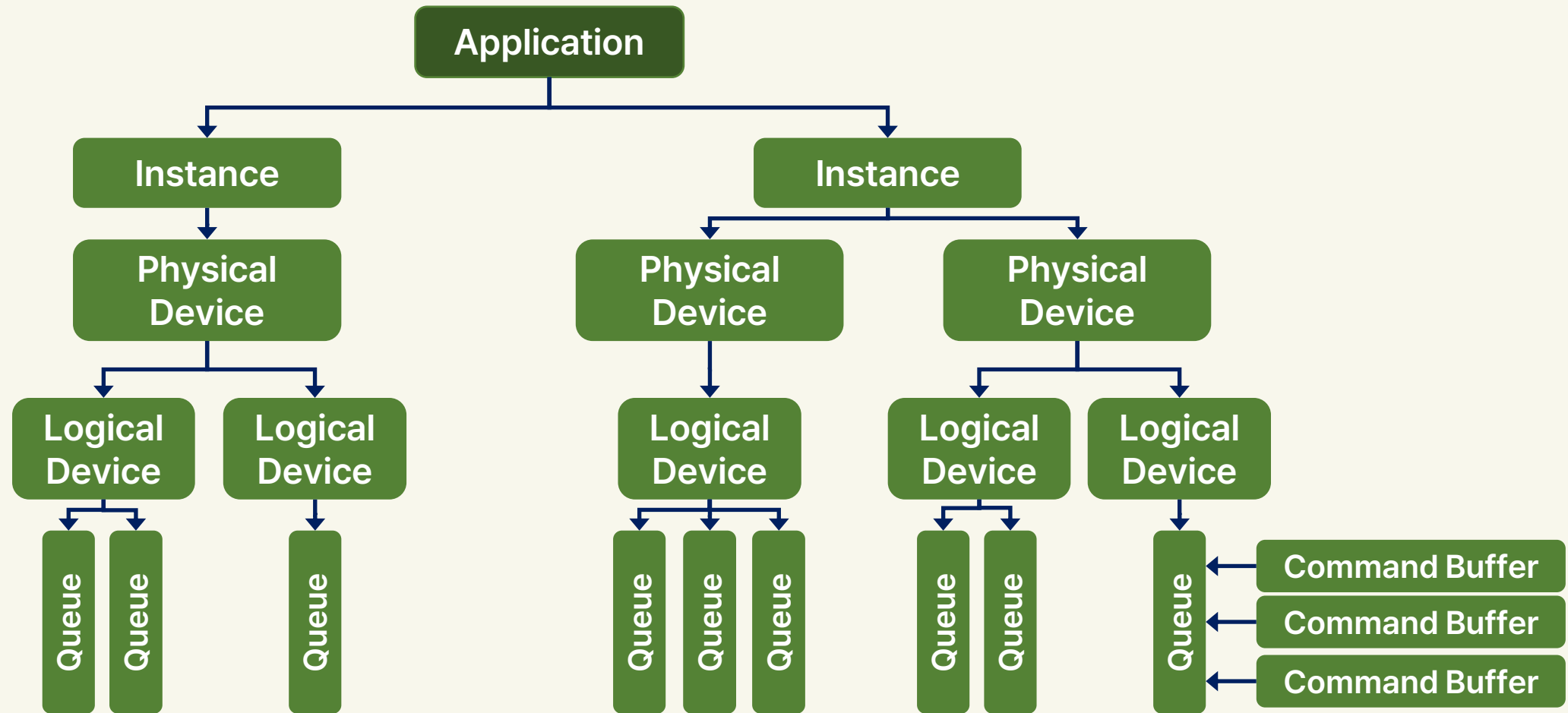
# Let's Set Up

---

- Check the triangle!

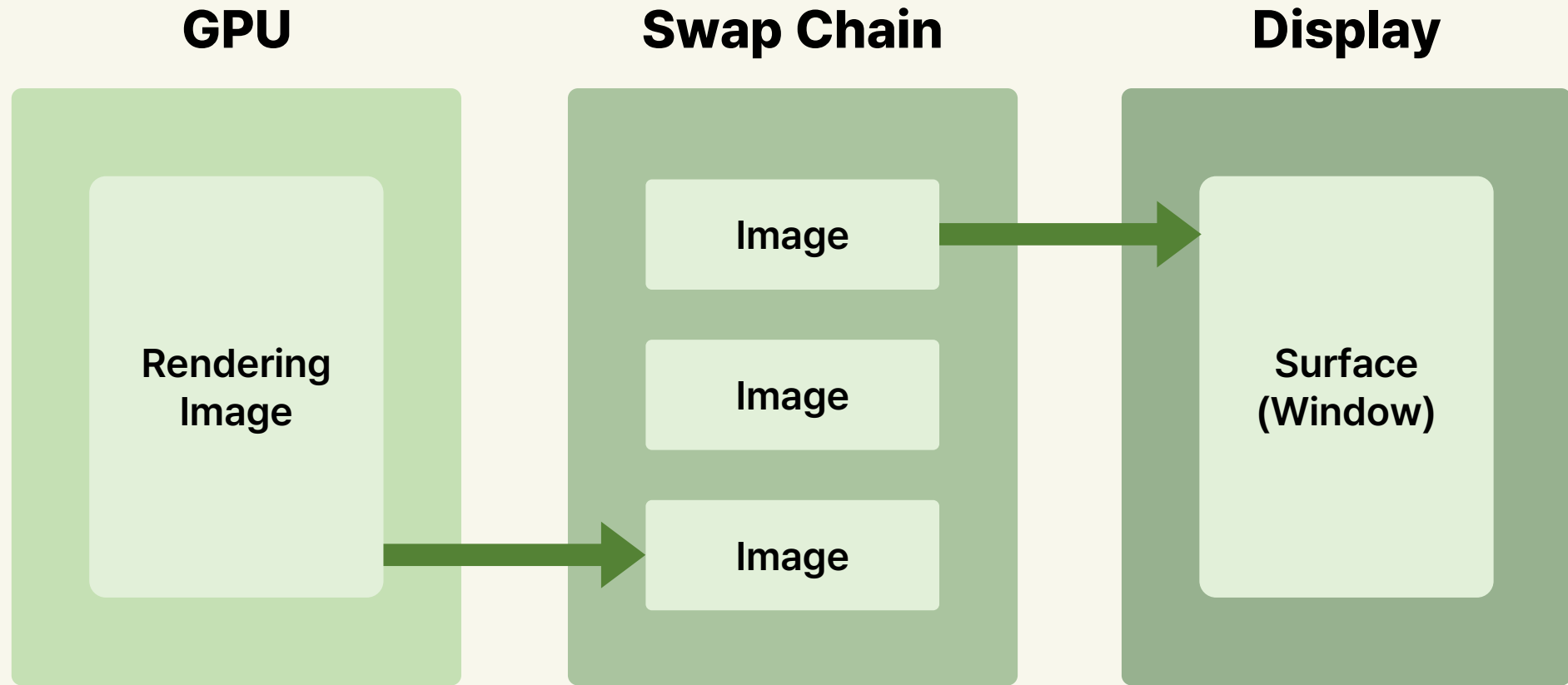


# Vulkan Instance and Device Hierarchy



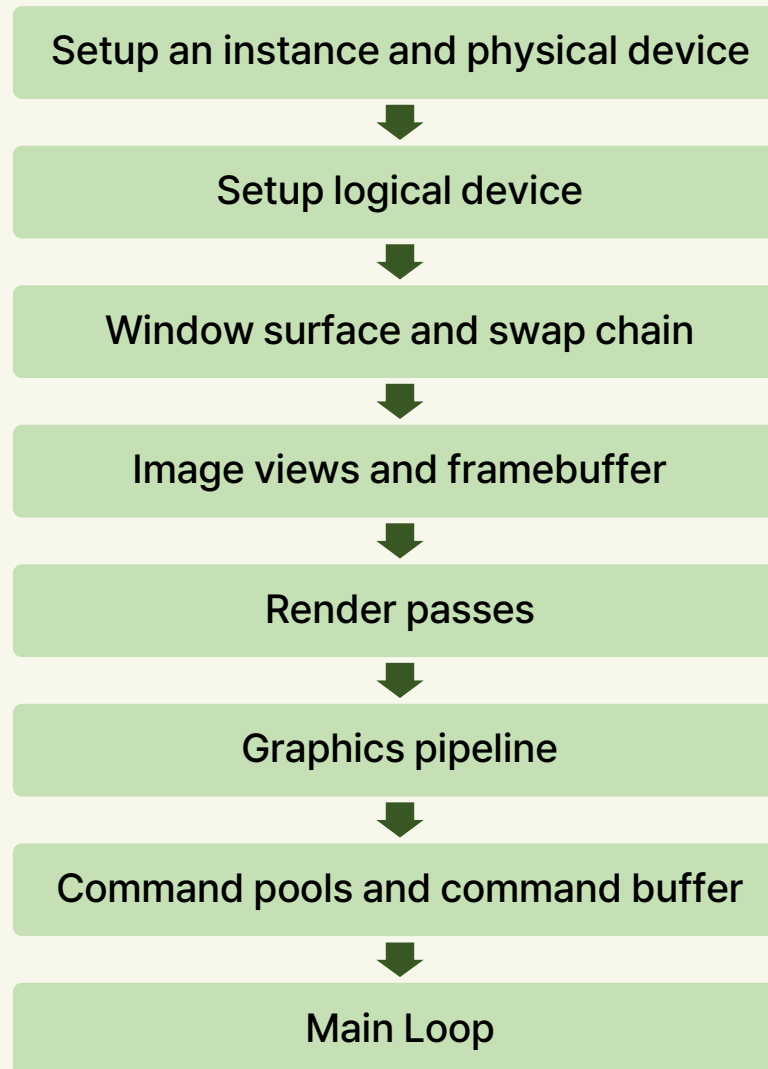
# Rendering to Surface Process

---



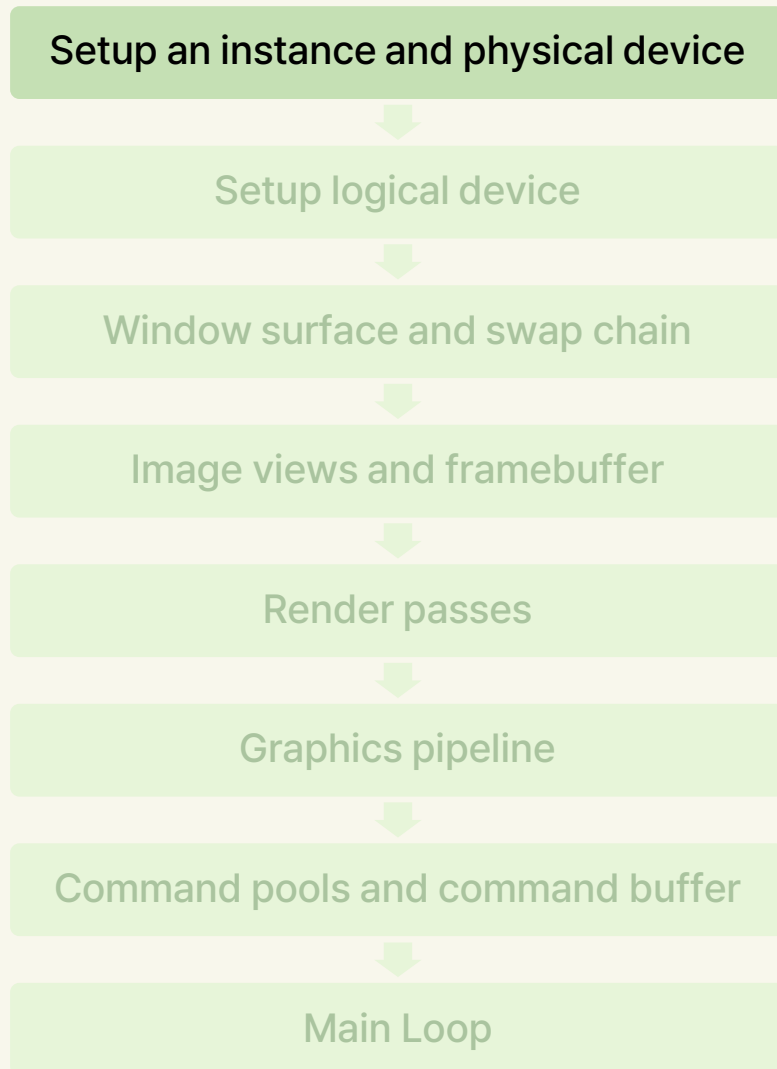
# Example of drawing a triangle

---



# Example of drawing a triangle

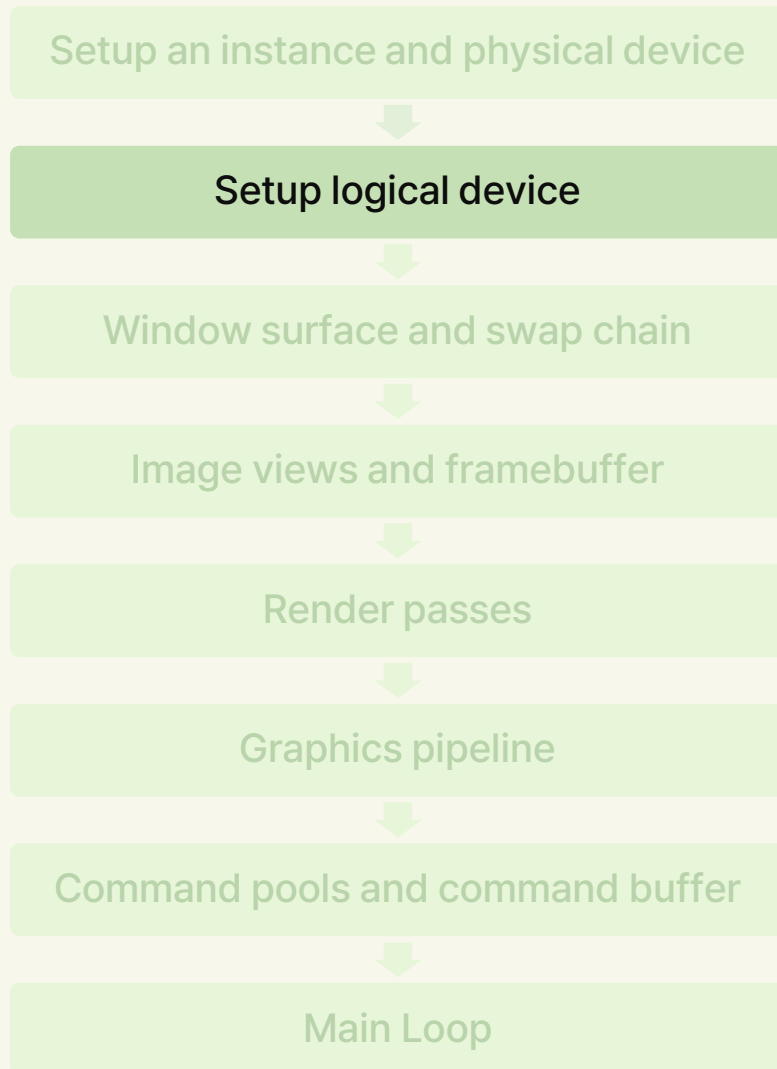
---



- **VkInstance** : Providing information on application and API extensions to be used
- **VkPhysicalDevice** : GPU(s) to be used
- Query on its capability and know about the optimization method to be used

# Example of drawing a triangle

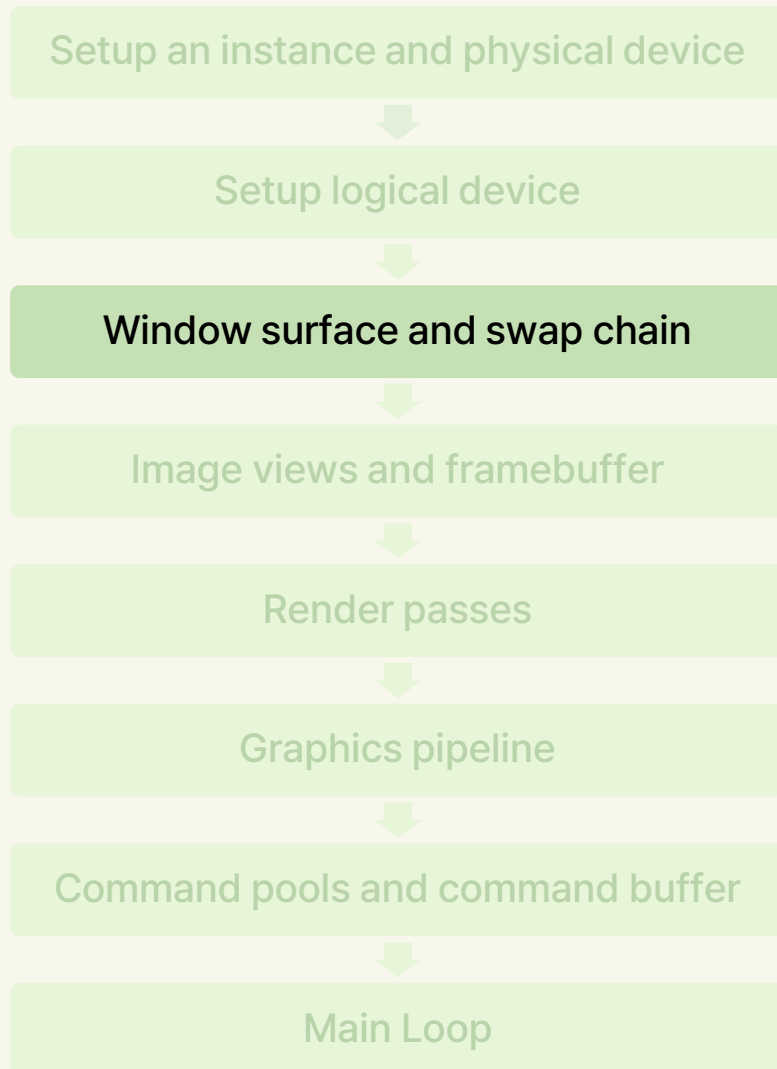
---



- **VkDevice** : Logical device to be used – specifying which the physical device features to be used
- **VkQueue** : Specifying data and command queues

# Example of drawing a triangle

---

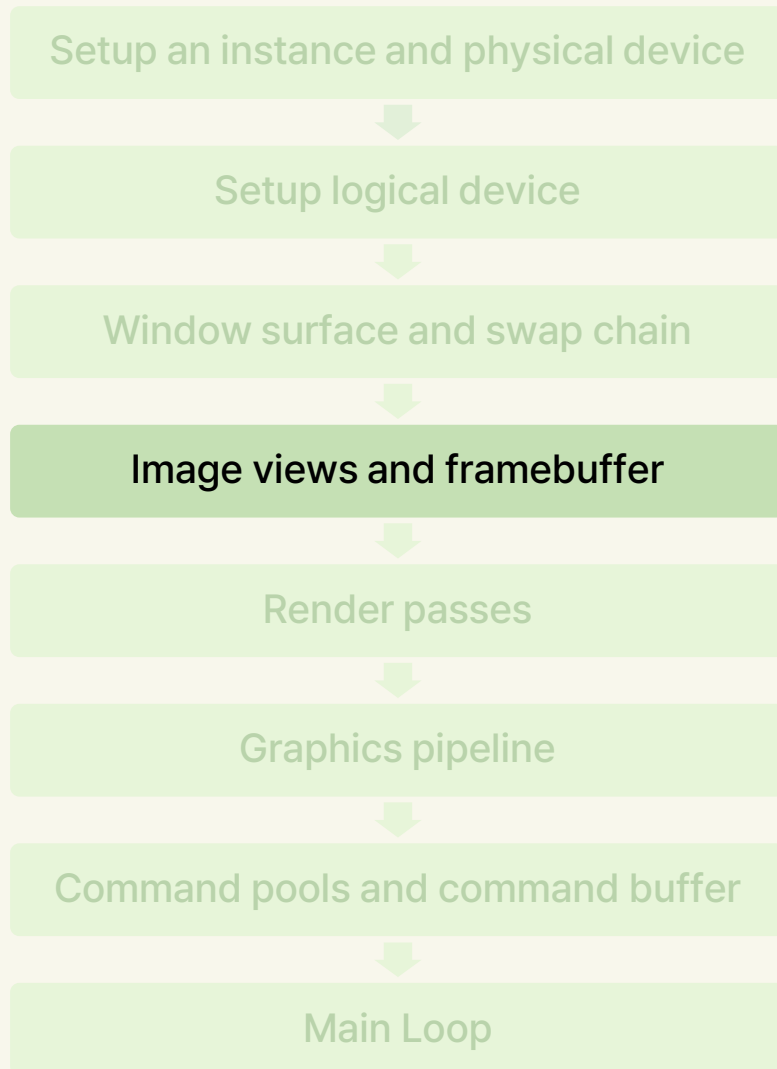


- **Create a window** : You may use GLFW, GLEW, GLUT etc...
- **VkSurfaceKHR** : Window surface to be rendered
- **VkSwapchainKHR** : Swap chain (e.g. double buffers)



# Example of drawing a triangle

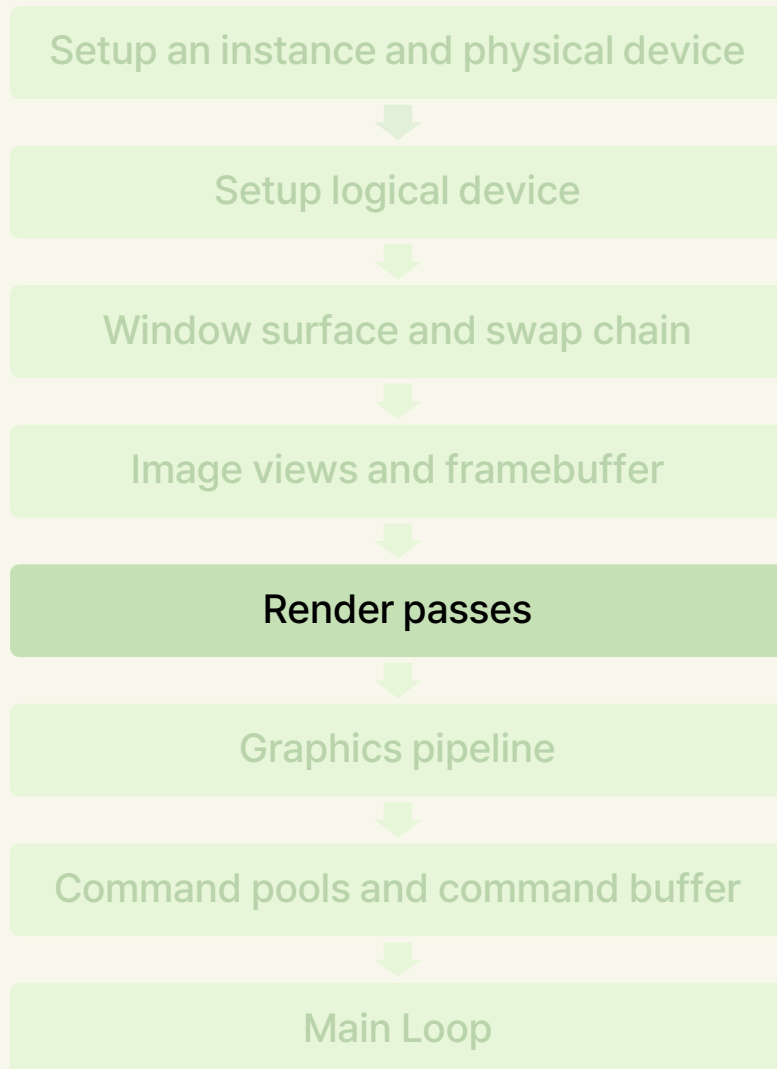
---



- **VkImageView** : Part of image to be used (e.g. for frame buffer)
- **VkFramebuffer** : Image views to be used for color, depth and/or stencil

# Example of drawing a triangle

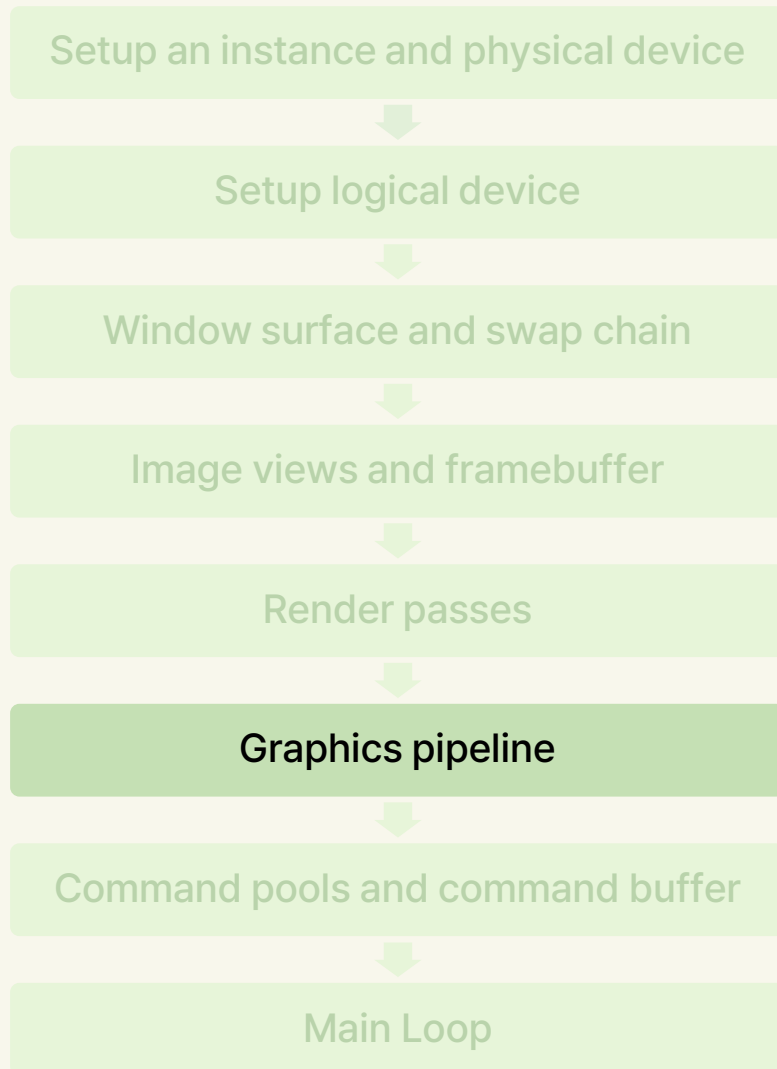
---



- Description on type of images to be used

# Example of drawing a triangle

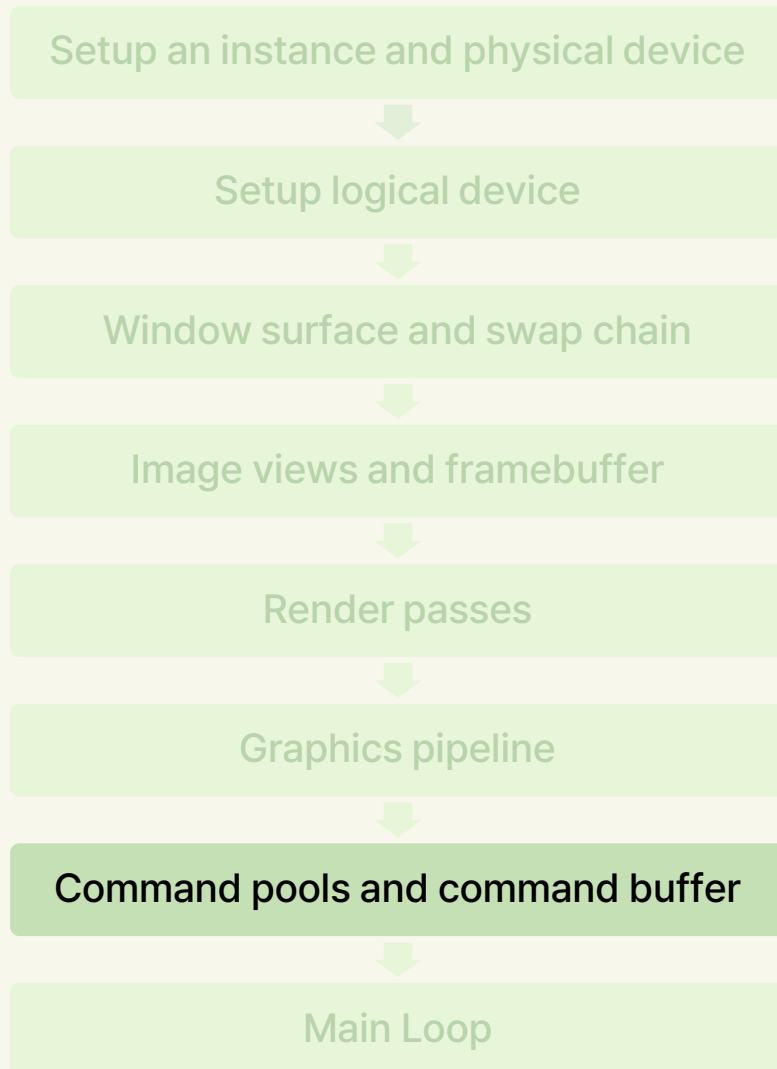
---



- **VkPipeline** : GPU status – viewport size, depth buffer operations, shaders, etc + binding to the render targets
  - **VkShaderMode** : Shader byte code

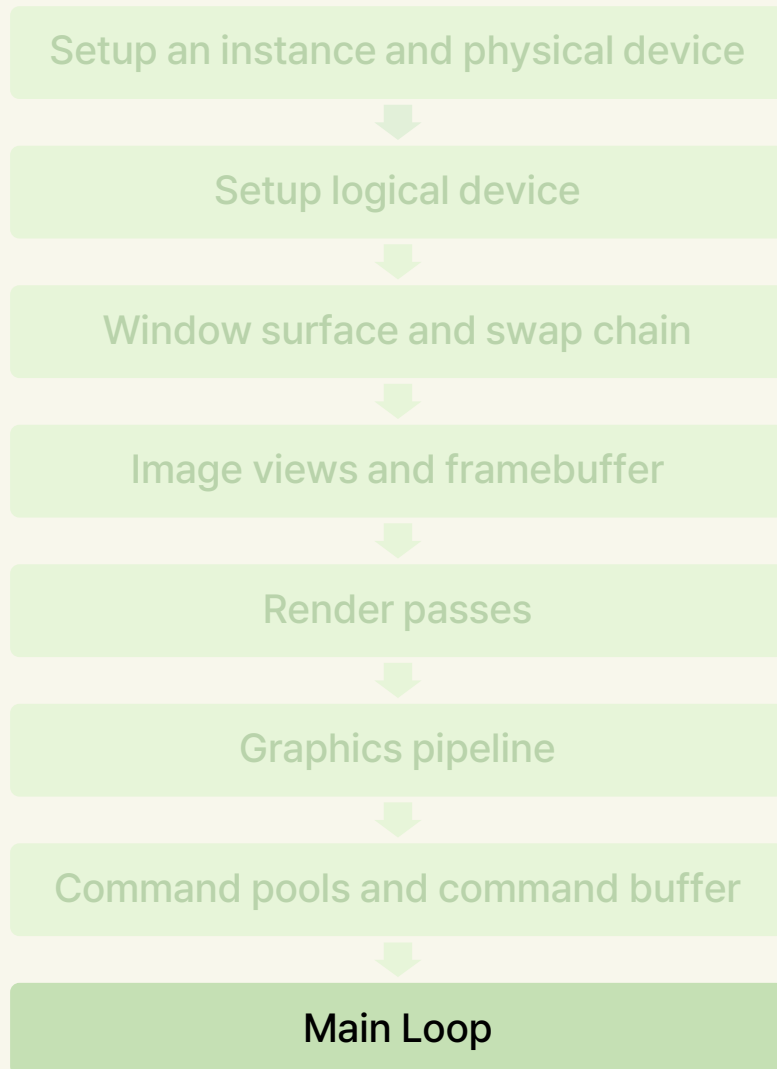
# Example of drawing a triangle

---

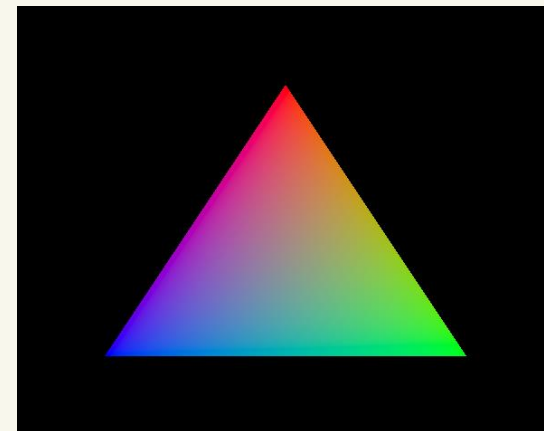


- **VkCommandBuffer** : Queue buffer for command
  - **VkCommandPool** : Pool where buffer could be allocated from
- **E.g**
  - Begin the render pass
  - Bind the graphics pipeline
  - Draw vertices
  - End the render pass

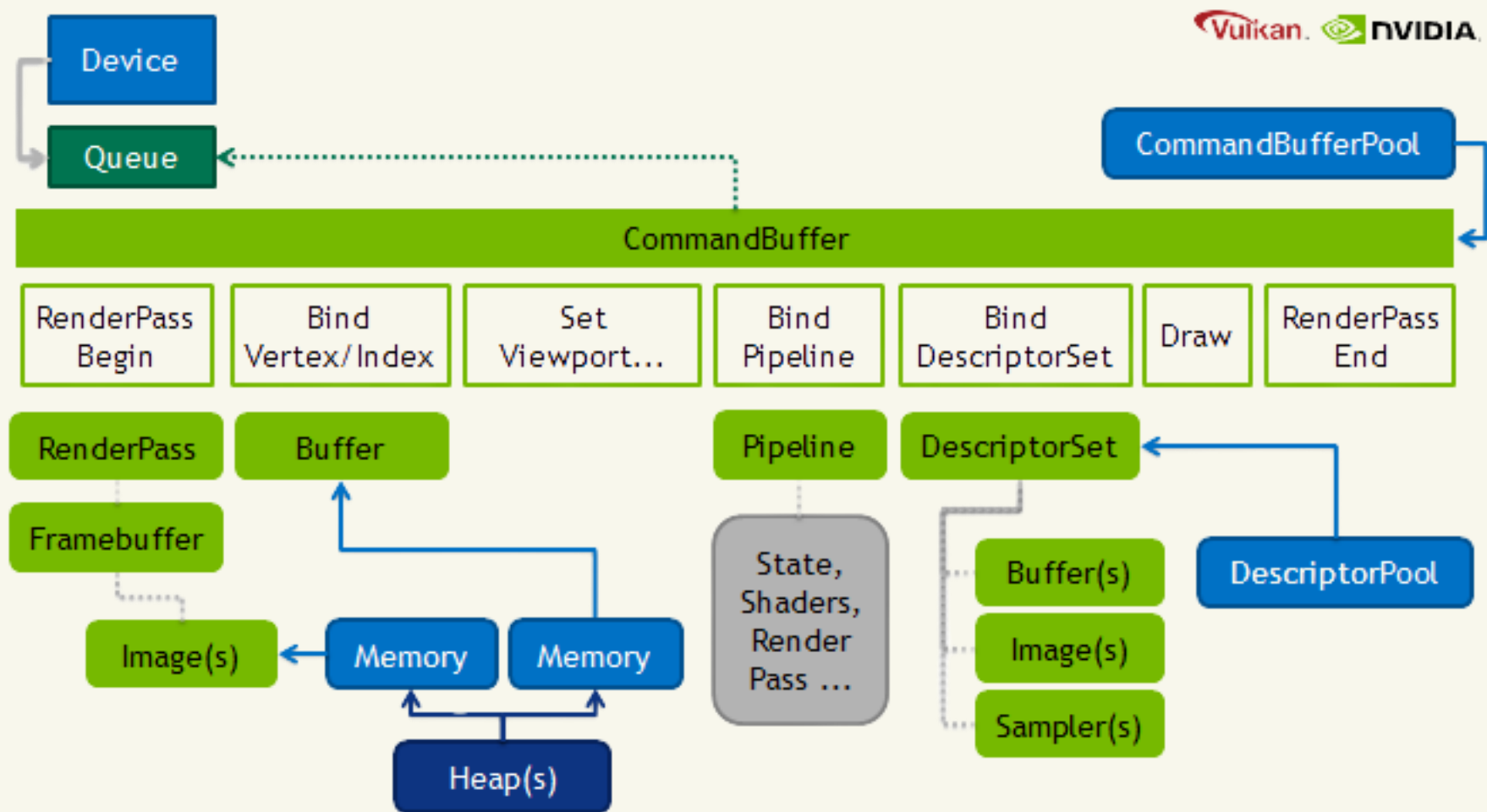
# Example of drawing a triangle



- **vkAcquireNextImageKHR** : Acquire an image from the swap chain
- **Select appropriate command buffer**
- **vkQueueSubmit** : Submit buffer and execute (asynchronous!)
- **vkQueuePresentKHR** : Return the created image to swap chain



# Vulkan Graphics Workflow Overview



# **01. Introduction to Vulkan & Fundamentals Setup**

# The first Vulkan example

---

- What you need for example
  - Vulkan SDK
  - GLFW
  - GLM
  - Create an empty project
- Add include directories for glm/glfw/vulkan
- Add library directories for vulkan/glfw
- Add vulkan-1.lib, glfw3.lib
- Setup compiler to support C++17 standard



# Create empty window

## Try to run

- Testing development environment

```
#define GLFW_INCLUDE_VULKAN
#include <GLFW/glfw3.h>
#define GLM_FORCE_RAIDIANs
#define GLM_FORCE_DEPTH_ZERO_TO_ONE
#include <glm/vec4.hpp>
#include <glm/mat4x4.hpp>

#include <iostream>
```

```
int main() {
    glfwInit();

    glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
    GLFWwindow* window = glfwCreateWindow(800, 600, "Vulkan window", nullptr, nullptr);
    uint32_t extensionCount = 0;
    vkEnumerateInstanceExtensionProperties(nullptr, &extensionCount, nullptr);

    std::cout << extensionCount << "extension supported\n";

    glm::mat4 matrix;
    glm::vec4 vec;
    auto test = matrix * vec;

    while (!glfwWindowShouldClose(window)) {
        glfwPollEvents();
    }

    glfwDestroyWindow(window);

    glfwTerminate();

    return 0;
}
```

# The Real Main Function

---

```
#include <vulkan/vulkan.h>
#include <iostream>
#include <stdexcept>
#include <cstdlib>

class HelloTriangleApplication {
public:
    void run() {
        initVulkan();
        mainLoop();
        cleanup();
    }
private:
    void initVulkan() {

    }

    void mainLoop() {

    }
}
```

```
void cleanup() {
}
;

int main()
{
    HelloTriangleApplication app;

    try
    {
        app.run();
    }
    catch(const std::exception& e)
    {
        std::cerr << e.what() << std::endl;
        return EXIT_FAILURE;
    }

    return EXIT_SUCCESS;
}
```

# Init Window

```
void initWindow()
{
    glfwInit();

    glfwWindowHint(GLFW_CLIENT_API, GLFW_NO_API);
    glfwWindowHint(GLFW_RESIZABLE, GLFW_FALSE);

    window = glfwCreateWindow(WIDTH, HEIGHT, "Vulkan", nullptr, nullptr);
}

void mainLoop()
{
    while(!glfwWindowShouldClose(window))
    {
        glfwPollEvents();
    }
}

void cleanup()
{
    glfwDestroyWindow(window);

    glfwTerminate();
}
```

- **initWindow():** Initializes GLFW and creates a window.
- **mainLoop():** Handles the main event and input loop.
- **cleanup():** Cleans up resources and closes the application.

# Create Instance

---

## Why Create an Instance in Vulkan?

- Establishes the link between your application and Vulkan
- Essential for initializing the Vulkan library

# Create Instance

## Why Create an Instance in Vulkan?

- Establishes the link between your application and Vulkan
- Essential for initializing the Vulkan library

- **VkApplicationInfo** : Optional / Information for optimizing applications
- **VkInstanceCreateInfo** : Mandatory / To tell the driver which extensions and/or validation layer to use

```
void createInstance() {  
  
    VkApplicationInfo applInfo{};  
    applInfo.sType = VK_STRUCTURE_TYPE_APPLICATION_INFO;  
    applInfo.pApplicationName = "Hello Triangle";  
    applInfo.applicationVersion = VK_MAKE_VERSION(1, 0, 0);  
    applInfo.pEngineName = "No Engine";  
    applInfo.engineVersion = VK_MAKE_VERSION(1, 0, 0);  
    applInfo.apiVersion = VK_API_VERSION_1_0;  
}  
  
VkInstanceCreateInfo createInfo{};  
createInfo.sType = VK_STRUCTURE_TYPE_INSTANCE_CREATE_INFO;  
createInfo.pApplicationInfo = &applInfo;  
  
uint32_t glfwExtensionCount = 0;  
const char** glfwExtensions;  
  
glfwExtensions = glfwGetRequiredInstanceExtensions(&glfwExtensionCount);  
  
createInfo.enabledExtensionCount = glfwExtensionCount;  
createInfo.ppEnabledExtensionNames = glfwExtensions;  
  
createInfo.enabledLayerCount = 0; Global Validation Layers
```

# Create Instance

- Create

Pointer for custom  
allocator

```
if(vkCreateInstance(&createInfo, nullptr, &instance) != VK_SUCCESS)
{
    throw std::runtime_error("failed to create instance!");
}
```

For MacOS

```
std::vector<const char*> requiredExtensions;

for(uint32_t i = 0; i < glfwExtensionCount; i++)
{
    requiredExtensions.emplace_back(glfwExtensions[i]);
}

requiredExtensions.emplace_back(VK_KHR_PORTABILITY_ENUMERATION_EXTENSION_NAME);

createInfo.flags |= VK_INSTANCE_CREATE_ENUMERATE_PORTABILITY_BIT_KHR;
createInfo.enabledExtensionCount = (uint32_t)requiredExtensions.size();
createInfo.ppEnabledExtensionNames = requiredExtensions.data();

if(vkCreateInstance(&createInfo, nullptr, &instance) != VK_SUCCESS)
{
    throw std::runtime_error("failed to create instance!");
}
```

# Create Instance

---

- **Check Extensions** : Optional / Check from the enumerated properties
- **vkEnumerateInstanceExtensionProperties** : Used to retrieve the list of supported extensions before creating an instance
- **VkExtensionProperties** : A data structure used to store information about Vulkan extensions

```
uint32_t extensionCount = 0;  
vkEnumerateInstanceExtensionProperties(nullptr, &extensionCount, nullptr);
```

```
std::vector<VkExtensionProperties> extensions(extensionCount);
```

```
vkEnumerateInstanceExtensionProperties(nullptr, &extensionCount,  
extensions.data());  
std::cout << "available extensions:\n";
```

```
for(const auto& extension : extensions)  
{  
    std::cout << '\t' << extension.extensionName << '\n';  
}
```

# Create Instance

---

```
void initVulkan()
{
    createInstance();
}

void cleanup()
{
    vkDestroyInstance(instance, nullptr);
    glfwDestroyWindow(window);
    glfwTerminate();
}

void mainLoop()
{
    while(!glfwWindowShouldClose(window))
    {
        glfwPollEvents();
    }
}

/* GLFWwindow* window; VkInstance instance; */
```



# Validation Layer

- **For debug:** Validate each function before run
- **Not Vulkan native but LunaG Vulkan SDK has additional features**
- **vkEnumerateInstanceLayerProperties** : List all of the available layers

```
const std::vector<const char*> validationLayers = {  
    "VK_LAYER_KHRONOS_validation";
```

```
#ifdef NDEBUG  
    const bool enableValidationLayers = false;  
#else  
    const bool enableValidationLayers = true;  
#endif
```

For Selective Setup

```
bool checkValidationLayerSupport()  
{  
    uint32_t layerCount;  
    vkEnumerateInstanceLayerProperties(&layerCount, nullptr);  
    std::vector<VkLayerProperties> availableLayers(layerCount);  
    vkEnumerateInstanceLayerProperties(&layerCount, availableLayers.data());  
  
    for (const char* layerName : validationLayers) {  
        bool layerFound = false;  
  
        for (const auto& layerProperties : availableLayers)  
        {  
            if (strcmp(layerName, layerProperties.layerName) == 0)  
            {  
                layerFound = true;  
                break;  
            }  
        }  
        if (!layerFound)  
        {  
            return false;  
        }  
    }  
    return true;  
}
```

# Validation Layer

---

- **For debug:** validate each function before run
- **Not Vulkan native but LunaG Vulkan SDK has additional features**

```
void createInstance() {  
    if(enableValidationLayers && !checkValidationLayerSupport()) {  
        throw std::runtime_error("validation layers requested, but not available!");  
    }  
    if(enableValidationLayers){  
        createInfo.enabledLayerCount = static_cast<uint32_t>(validationLayers.size());  
        createInfo.ppEnabledLayerNames = validationLayers.data();  
    }  
    else {  
        createInfo.enabledLayerCount = 0;  
    }  
}
```

# Validation Layer

---

- **Message Callback for Debug**
- **getRequiredExtensions()** : Return the required list of extensions based on whether validation layers are enabled or not

```
std::vector<const char*> getRequiredExtensions()
{
    uint32_t    glfwExtensionCount = 0;
    const char** glfwExtensions;
    glfwExtensions = glfwGetRequiredInstanceExtensions(&glfwExtensionCount);

    std::vector<const char*> extensions(glfwExtensions, glfwExtensions + glfwExtensionCount);
    if(enableValidationLayers)
    {
        extensions.push_back(VK_EXT_DEBUG_UTILS_EXTENSION_NAME);
    }
    return extensions;
}
```

- **Add into createInstance**

```
auto extensions = getRequiredExtensions();
createInfo.enabledExtensionCount = static_cast<uint32_t>(extensions.size());
createInfo.ppEnabledExtensionNames = extensions.data();
```

# Validation Layer

---

- **Debug Callback**

```
static VKAPI_ATTR VkBool32 VKAPI_CALL debugCallback(  
    VkDebugUtilsMessageSeverityFlagBitsEXT messageSeverity,  
    VkDebugUtilsMessageTypeFlagsEXT messageType,  
    const VkDebugUtilsMessengerCallbackDataEXT* pCallbackData,  
    void* pUserData) {  
  
    std::cerr << "validation layer: " << pCallbackData->pMessage << std::endl;  
  
    return VK_FALSE;  
}
```

- **VkDebugUtilsMessageSeverityFlagBitsEXT** : Bitmask specifying which severities of events cause a debug messenger callback
  - **VK\_DEBUG\_UTILS\_MESSAGE\_SEVERITY\_VERBOSE\_BIT\_EXT**: Diagnostic message
  - **VK\_DEBUG\_UTILS\_MESSAGE\_SEVERITY\_INFO\_BIT\_EXT**: Informational message like the creation of a resource
  - **VK\_DEBUG\_UTILS\_MESSAGE\_SEVERITY\_WARNING\_BIT\_EXT**: Message about behavior that is not necessarily an error, but very likely a bug in your application
  - **VK\_DEBUG\_UTILS\_MESSAGE\_SEVERITY\_ERROR\_BIT\_EXT**: Message about behavior that is invalid and may cause crashes

# Validation Layer

---

- **Debug Callback**

```
static VKAPI_ATTR VkBool32 VKAPI_CALL debugCallback(  
    VkDebugUtilsMessageSeverityFlagBitsEXT messageSeverity,  
    VkDebugUtilsMessageTypeFlagsEXT messageType,  
    const VkDebugUtilsMessengerCallbackDataEXT* pCallbackData,  
    void* pUserData) {  
  
    std::cerr << "validation layer: " << pCallbackData->pMessage << std::endl;  
  
    return VK_FALSE;  
}
```

- **VkDebugUtilsMessageTypeFlagsEXT** : Bitmask of VkDebugUtilsMessageTypeFlagBitsEXT
  - **VK\_DEBUG\_UTILS\_MESSAGE\_TYPE\_GENERAL\_BIT\_EXT**: Some event has happened that is unrelated to the specification or performance
  - **VK\_DEBUG\_UTILS\_MESSAGE\_TYPE\_VALIDATION\_BIT\_EXT**: Something has happened that violates the specification or indicates a possible mistake
  - **VK\_DEBUG\_UTILS\_MESSAGE\_TYPE\_PERFORMANCE\_BIT\_EXT**: Potential non-optimal use of Vulkan

# Validation Layer

---

- **Debug Callback**

```
static VKAPI_ATTR VkBool32 VKAPI_CALL debugCallback(  
    VkDebugUtilsMessageSeverityFlagBitsEXT messageSeverity,  
    VkDebugUtilsMessageTypeFlagsEXT messageType,  
    const VkDebugUtilsMessengerCallbackDataEXT* pCallbackData,  
    void* pUserData) {  
  
    std::cerr << "validation layer: " << pCallbackData->pMessage << std::endl;  
  
    return VK_FALSE;  
}
```

- **VkDebugUtilsMessengerCallbackDataEXT** : Structure specifying parameters returned to the callback
  - **pMessage**: The debug message as a null-terminated string
  - **pObjects**: Array of Vulkan object handles related to the message
  - **objectCount**: Number of objects in array

**Solely for validating the validation layers, always return VK\_FALSE**

# Validation Layer

---

- Debug Utils

```
VkDebugUtilsMessengerCreateInfoEXT createInfo{};
createInfo.sType = VK_STRUCTURE_TYPE_DEBUG_UTILS_MESSENGER_CREATE_INFO_EXT;
createInfo.messageSeverity = VK_DEBUG_UTILS_MESSAGE_SEVERITY_VERBOSE_BIT_EXT | VK_DEBUG_UTILS_MESSAGE_SEVERITY_WARNING_BIT_EXT |
VK_DEBUG_UTILS_MESSAGE_SEVERITY_ERROR_BIT_EXT;
createInfo.messageType = VK_DEBUG_UTILS_MESSAGE_TYPE_GENERAL_BIT_EXT | VK_DEBUG_UTILS_MESSAGE_TYPE_VALIDATION_BIT_EXT |
VK_DEBUG_UTILS_MESSAGE_TYPE_PERFORMANCE_BIT_EXT;
createInfo.pfnUserCallback = debugCallback;
createInfo.pUserData = nullptr; // Optional
```

- **VkDebugUtilsMessengerCreateInfoEXT**: A structure for debug messenger creation.
  - **sType**: Specifies the type of the structure.
  - **messageSeverity**: Defines the severity of the debug messages.
  - **messageType**: Indicates the type of debug messages.
  - **pfnUserCallback**: A user-provided callback function.
  - **pUserData**: User data passed to the callback. (Optional)

# Validation Layer

- Debug Utils

```
VkResult CreateDebugUtilsMessengerEXT(VkInstance instance, const VkDebugUtilsMessengerCreateInfoEXT* pCreateInfo, const
VkAllocationCallbacks* pAllocator, VkDebugUtilsMessengerEXT* pDebugMessenger) {
    auto func = (PFN_vkCreateDebugUtilsMessengerEXT)vkGetInstanceProcAddr(instance, "vkCreateDebugUtilsMessengerEXT");
    if (func != nullptr) {
        return func(instance, pCreateInfo, pAllocator, pDebugMessenger);
    }
    else {
        return VK_ERROR_EXTENSION_NOT_PRESENT;
    }
}

if (CreateDebugUtilsMessengerEXT(instance, &createInfo, nullptr, &debugMessenger) != VK_SUCCESS) {
    throw std::runtime_error("failed to set up debug messenger!");
}

void DestroyDebugUtilsMessengerEXT(VkInstance instance, VkDebugUtilsMessengerEXT debugMessenger, const VkAllocationCallbacks * pAllocator)
{
    auto func = (PFN_vkDestroyDebugUtilsMessengerEXT)vkGetInstanceProcAddr(instance, "vkDestroyDebugUtilsMessengerEXT");
    if (func != nullptr) {
        func(instance, debugMessenger, pAllocator);
    }
}
```

- **vkGetInstanceProcAddr** : DebugLook up its address using vkGetInstanceProcAddr ourselves



# Physical Device

---

- **Graphics Card Selection:** Search and choose a suitable graphics card that supports desired features.
  - \* **Note:** Multiple graphics cards can be used simultaneously.

```
void initVulkan() {  
    createInstance();  
    setupDebugMessenger();  
    pickPhysicalDevice();  
}  
  
void pickPhysicalDevice() {  
  
}  
VkPhysicalDevice physicalDevice = VK_NULL_HANDLE;
```

# Physical Device

- Enumerate and Evaluate

```
uint32_t deviceCount = 0;
vkEnumeratePhysicalDevices(instance, &deviceCount, nullptr);
if (deviceCount == 0) {
    throw std::runtime_error("failed to find GPUs with Vulkan support!");
}
std::vector<VkPhysicalDevice> devices(deviceCount);
vkEnumeratePhysicalDevices(instance, &deviceCount, devices.data());
for (const auto& device : devices) {
    if (isDeviceSuitable(device)) {
        physicalDevice = device;
        break;
    }
}

if (physicalDevice == VK_NULL_HANDLE) {
    throw std::runtime_error("failed to find a suitable GPU!");
}
```

- Check the physical devices that meet the requirements

- IsSuitable example

```
bool isDeviceSuitable(VkPhysicalDevice device) {
    VkPhysicalDeviceProperties deviceProperties;
    VkPhysicalDeviceFeatures deviceFeatures;
    vkGetPhysicalDeviceProperties(device, &deviceProperties);
    vkGetPhysicalDeviceFeatures(device, &deviceFeatures);

    return deviceProperties.deviceType ==
           VK_PHYSICAL_DEVICE_TYPE_DISCRETE_GPU &&
           deviceFeatures.geometryShader;
}
```

- vkGetPhysicalDeviceProperties** : Basic device properties can be queried (e.g. the name, type and supported Vulkan version)
- vkGetPhysicalDeviceFeatures** : The support for optional features can be queried ( e.g. texture compression, 64 bit floats and multi viewport rendering (useful for VR))
- Score each device to select the appropriate device

# Physical Device & Queue Families

- Queue Families
  - Queue for command
  - Difference command set for different families
    - Eg: Compute or Memory or ...

```
struct QueueFamilyIndices {
    uint32_t graphicsFamily;
};

QueueFamilyIndices findQueueFamilies(VkPhysicalDevice device) {
    QueueFamilyIndices indices;

    ...

    return indices;
}

uint32_t queueFamilyCount = 0;
vkGetPhysicalDeviceQueueFamilyProperties(device, &queueFamilyCount, nullptr);
std::vector<VkQueueFamilyProperties> queueFamilies(queueFamilyCount);
vkGetPhysicalDeviceQueueFamilyProperties(device, &queueFamilyCount,
queueFamilies.data());
```

- **vkGetPhysicalDeviceQueueFamilyProperties :**  
VkPhysicalDevice                      physicalDevice,  
uint32\_t\*                                pQueueFamilyPropertyCount,  
VkQueueFamilyProperties\*                pQueueFamilyProperties);

- Find a queue family that supports **VK\_QUEUE\_GRAPHICS\_BIT**

```
int i = 0;
for (const auto& queueFamily : queueFamilies) {
    if (queueFamily.queueFlags & VK_QUEUE_GRAPHICS_BIT)
    {
        indices.graphicsFamily = i;
    }

    i++;
}

bool isDeviceSuitable(VkPhysicalDevice device) {
    QueueFamilyIndices indices = findQueueFamilies(device);

    return indices.graphicsFamily.has_value();
}
```

- **isDeviceSuitable :** Verify that a given physical device is capable of handling graphical commands.

# Logical Device and Queue

---

- **Logical Device** : Interface with the selected physical device
- **Possible to create multiple logical devices from one physical device**

```
VkDevice device;  
  
void initVulkan() {  
    createInstance();  
    setupDebugMessenger();  
    pickPhysicalDevice();  
    createLogicalDevice();  
}  
  
void createLogicalDevice() {  
  
}
```

# Logical Device and Queue

---

- **Create Queue**

```
QueueFamilyIndices indices = findQueueFamilies(physicalDevice);

VkDeviceQueueCreateInfo queueCreateInfo{};
queueCreateInfo.sType = VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO;
queueCreateInfo.queueFamilyIndex = indices.graphicsFamily.value();
queueCreateInfo.queueCount = 1;

float queuePriority = 1.0f;
queueCreateInfo.pQueuePriorities = &queuePriority;
```

- **Queue will be automatically created once the logical device is created**

```
VkQueue graphicsQueue;

vkGetDeviceQueue(device, indices.graphicsFamily.value(), 0, &graphicsQueue);
```

- **VkDeviceQueueCreateInfo**: Structure for storing queue generation information
- **vkGetDeviceQueue function**: Retrieves queue handles for each queue family.
  - Parameters : Logical device, Queue family, Queue index, Pointer to queue handle variable.

# Logical Device and Queue

- **Create Logical Device**

```
VkPhysicalDeviceFeatures deviceFeatures{};

VkDeviceCreateInfo createInfo{};
createInfo.sType = VK_STRUCTURE_TYPE_DEVICE_CREATE_INFO;

createInfo.pQueueCreateInfos = &queueCreateInfo;
createInfo.queueCreateInfoCount = 1;

createInfo.pEnabledFeatures = &deviceFeatures;

createInfo.enabledExtensionCount = 0;

if (enableValidationLayers) {
    createInfo.enabledLayerCount = static_cast<uint32_t>(validationLayers.size());
    createInfo.ppEnabledLayerNames = validationLayers.data();
}
else {
    createInfo.enabledLayerCount = 0;
}

if (vkCreateDevice(physicalDevice, &createInfo, nullptr, &device) != VK_SUCCESS)
{
    throw std::runtime_error("failed to create logical device!");
}
```

- **VkPhysicalDeviceFeatures** : Initializes a structure to store the features of supported physical devices.
- **pQueueCreateInfos** & **queueCreateInfoCount**: Specifies the information and number of queues to be created in the logical device.
- **pEnabledFeatures**: Specifies the device features you want to use.
- **enabledExtensionCount**: Specifies the number of extensions to enable
- **enabledLayerCount** & **ppEnabledLayerNames**: Specifies the number of verification layers to enable and their names

# Window surface creation

- Vulkan relies on WSI extensions like `VK_KHR_surface` to interface with window systems for rendering to the screen.

```
VkSurfaceKHR surface;
```

- To update in order to access native platform functions.

```
#define VK_USE_PLATFORM_WIN32_KHR
#define GLFW_INCLUDE_VULKAN
#include <GLFW/glfw3.h>
#define GLFW_EXPOSE_NATIVE_WIN32
#include <GLFW/glfw3native.h>
```

```
VkWin32SurfaceCreateInfoKHR createInfo{};
createInfo.sType = VK_STRUCTURE_TYPE_WIN32_SURFACE_CREATE_INFO_KHR;
createInfo.hwnd = glfwGetWin32Window(window);
createInfo.hinstance = GetModuleHandle(nullptr);
```

- **VK\_USE\_PLATFORM\_WIN32\_KHR**: Indicates that you want to use the Vulkan extension for the Windows platform.
- **GLFW\_INCLUDE\_VULKAN** & **GLFW\_EXPOSE\_NATIVE\_WIN32**: Definitions to include Vulkan and Windows specific features while using the GLFW library.
- **VkWin32SurfaceCreateInfoKHR** : Initializes a structure for creating a Vulkan surface on Windows platforms.
- **sType**: Specifies the type of this structure.
- **hwnd**: Gets the handle of a window created using GLFW.
- **hinstance**: Gets the instance handle of the current process.

# Window surface creation

---

```
if (vkCreateWin32SurfaceKHR(instance, &createInfo, nullptr, &surface) !=  
VK_SUCCESS) {  
    throw std::runtime_error("failed to create window surface!");  
}
```

```
void createSurface() {  
    if (glfwCreateWindowSurface(instance, window, nullptr, &surface) != VK_SUCCESS) {  
        throw std::runtime_error("failed to create window surface!");  
    }  
}
```

- **vkCreateWin32SurfaceKHR** : Create a Surface
  - Instance, surface creation details, custom allocators and the variable for the surface handle to be stored in.
- **GLFW call parameters** : VkInstance, GLFW window pointer, custom allocators and pointer to VkSurfaceKHR variable



# Present Queue

```
struct QueueFamilyIndices {  
    std::optional<uint32_t> graphicsFamily;  
    std::optional<uint32_t> presentFamily;  
  
    bool isComplete() {  
        return graphicsFamily.has_value() && presentFamily.has_value();  
    }  
};
```

```
VkBool32 presentSupport = false;  
    vkGetPhysicalDeviceSurfaceSupportKHR(device, i, surface, &presentSupport);  
  
    if (presentSupport) {  
        indices.presentFamily = i;  
    }
```

- **graphicsFamily**: Store the index of the queue family that can process graphics commands
- **presentFamily**: Store the index of the queue family that can handle presentation commands
- **isComplete()**: Function to check if both graphicsFamily and presentFamily have valid values (values are set)

- **vkGetPhysicalDeviceSurfaceSupportKHR**  
Parameters : physical device, queue family index and surface

# Present Queue

- Modifying the logical device creation procedure to create the presentation queue and retrieve the VkQueue handle.

```
VkQueue presentQueue;
```

```
QueueFamilyIndices indices = findQueueFamilies(physicalDevice);

std::vector<VkDeviceQueueCreateInfo> queueCreateInfos;
std::set<uint32_t> uniqueQueueFamilies = { indices.graphicsFamily.value(),
indices.presentFamily.value() };

float queuePriority = 1.0f;
for (uint32_t queueFamily : uniqueQueueFamilies) {
    VkDeviceQueueCreateInfo queueCreateInfo{};
    queueCreateInfo.sType =
VK_STRUCTURE_TYPE_DEVICE_QUEUE_CREATE_INFO;
    queueCreateInfo.queueFamilyIndex = queueFamily;
    queueCreateInfo.queueCount = 1;
    queueCreateInfo.pQueuePriorities = &queuePriority;
    queueCreateInfos.push_back(queueCreateInfo);
}
```

```
createInfo.queueCreateInfoCount = static_cast<uint32_t>(queueCreateInfos.size());
createInfo.pQueueCreateInfos = queueCreateInfos.data();
```

```
vkGetDeviceQueue(device, indices.presentFamily.value(), 0, &presentQueue);
```

- **std::vector<VkDeviceQueueCreateInfo> queueCreateInfos:** Stores queue creation information.
- **std::set<uint32\_t> uniqueQueueFamilies:** A set to store indices of graphics and presentation queue families without duplication.
- **createInfo.queueCreateInfoCount & createInfo.pQueueCreateInfos:** Set the number and pointer of queue creation information.

# Swap Chain

---

- **Swapchain** : Proper device setting
  - A queue of images that are waiting to be presented to the screen
- 
- **VK\_KHR\_SWAPCHAIN\_EXTENSION\_NAME** : Detect spelling errors

```
const std::vector<const char*> deviceExtensions = {  
    VK_KHR_SWAPCHAIN_EXTENSION_NAME  
};
```

```
bool isDeviceSuitable(VkPhysicalDevice device) {  
    QueueFamilyIndices indices = findQueueFamilies(device);  
  
    bool extensionsSupported = checkDeviceExtensionSupport(device);  
  
    return indices.isComplete() && extensionsSupported && swapChainAdequate;  
}
```

# Swap Chain

---

- Verify that the physical device supports the required queue families and extensions
- Determine if the device is appropriate for use

```
bool checkDeviceExtensionSupport(VkPhysicalDevice device) {
    uint32_t extensionCount;
    vkEnumerateDeviceExtensionProperties(device, nullptr, &extensionCount, nullptr);

    std::vector<VkExtensionProperties> availableExtensions(extensionCount);
    vkEnumerateDeviceExtensionProperties(device, nullptr, &extensionCount, availableExtensions.data());

    std::set<std::string> requiredExtensions(deviceExtensions.begin(), deviceExtensions.end());

    for (const auto& extension : availableExtensions) {
        requiredExtensions.erase(extension.extensionName);
    }
    return requiredExtensions.empty();
}
```

# Swap Chain

- Query the swap chain's supported formats and presentation modes

```
SwapChainSupportDetails querySwapChainSupport(VkPhysicalDevice device) {
    SwapChainSupportDetails details;

    vkGetPhysicalDeviceSurfaceCapabilitiesKHR(device, surface, &details.capabilities);

    uint32_t formatCount;
    vkGetPhysicalDeviceSurfaceFormatsKHR(device, surface, &formatCount, nullptr);

    if (formatCount != 0) {
        details.formats.resize(formatCount);
        vkGetPhysicalDeviceSurfaceFormatsKHR(device, surface, &formatCount, details.formats.data());
    }

    uint32_t presentModeCount;
    vkGetPhysicalDeviceSurfacePresentModesKHR(device, surface, &presentModeCount, nullptr);

    if (presentModeCount != 0) {
        details.presentModes.resize(presentModeCount);
        vkGetPhysicalDeviceSurfacePresentModesKHR(device, surface, &presentModeCount, details.presentModes.data());
    }
}
```

- Returned into a single `VkSurfaceCapabilitiesKHR` struct
- `vkGetPhysicalDeviceSurfaceCapabilitiesKHR` : Use the specified `VkPhysicalDevice` and `VkSurfaceKHR`

# Swap Chain

---

- Ensure swap chain support is adequate

```
bool swapChainAdequate = false;
if (extensionsSupported) {
    SwapChainSupportDetails swapChainSupport =
querySwapChainSupport(device);
    swapChainAdequate = !swapChainSupport.formats.empty()
&& !swapChainSupport.presentModes.empty();
}

return indices.isComplete() && extensionsSupported && swapChainAdequate;
```

# Swap Chain

---

- **Choosing the right surface**

```
VkSurfaceFormatKHR chooseSwapSurfaceFormat(const std::vector<VkSurfaceFormatKHR>& availableFormats) {  
    for (const auto& availableFormat : availableFormats) {  
        if (availableFormat.format == VK_FORMAT_B8G8R8A8_SRGB && availableFormat.colorSpace == VK_COLOR_SPACE_SRGB_NONLINEAR_KHR) {  
            return availableFormat;  
        }  
    }  
  
    return availableFormats[0];  
}
```

- **VkSurfaceFormatKHR** : Structure describing a supported swapchain format-color space pair

- **And Presentation mode**

- **IMMEDIATE / FIFO / FIFO\_RELAXED / MAILBOX**

```
VkPresentModeKHR chooseSwapPresentMode(const std::vector<VkPresentModeKHR>& availablePresentModes) {  
    return VK_PRESENT_MODE_FIFO_KHR;  
}
```

- **VkPresentModeKHR** : Presentation mode supported for a surface

# Swap Chain

- Choose swap extent : resolution

```
VkExtent2D chooseSwapExtent(const VkSurfaceCapabilitiesKHR& capabilities) {  
    if (capabilities.currentExtent.width != std::numeric_limits<uint32_t>::max()) {  
        return capabilities.currentExtent;  
    }  
    else {  
        int width, height;  
        glfwGetFramebufferSize(window, &width, &height);  
  
        VkExtent2D actualExtent = {  
            static_cast<uint32_t>(width),  
            static_cast<uint32_t>(height)  
        };  
  
        actualExtent.width = std::clamp(actualExtent.width, capabilities.minImageExtent.width, capabilities.maxImageExtent.width);  
        actualExtent.height = std::clamp(actualExtent.height, capabilities.minImageExtent.height, capabilities.maxImageExtent.height);  
  
        return actualExtent;  
    }  
}
```

- **VkSurfaceCapabilitiesKHR** : Contains functions for the surface of the swap chain
- **VkExtent2D** : Structure specifying a two-dimensional extent



# Swap Chain

- **Create Swapchain**

```
SwapChainSupportDetails swapChainSupport = querySwapChainSupport(physicalDevice);

VkSurfaceFormatKHR surfaceFormat = chooseSwapSurfaceFormat(swapChainSupport.formats);
VkPresentModeKHR presentMode = chooseSwapPresentMode(swapChainSupport.presentModes);
VkExtent2D extent = chooseSwapExtent(swapChainSupport.capabilities);

uint32_t imageCount = swapChainSupport.capabilities.minImageCount + 1;
if (swapChainSupport.capabilities.maxImageCount > 0 && imageCount > swapChainSupport.capabilities.maxImageCount) {
    imageCount = swapChainSupport.capabilities.maxImageCount;
}

VkSwapchainCreateInfoKHR createInfo{};
createInfo.sType = VK_STRUCTURE_TYPE_SWAPCHAIN_CREATE_INFO_KHR;
createInfo.surface = surface;

createInfo.minImageCount = imageCount;
createInfo.imageFormat = surfaceFormat.format;
createInfo.imageColorSpace = surfaceFormat.colorSpace;
createInfo.imageExtent = extent;
createInfo.imageArrayLayers = 1;
createInfo.imageUsage = VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT;
```

- **imageArrayLayers** : Always 1 (unless you are developing a stereoscopic 3D application)
- **imageUsage** : Specify what operation to use the image in the swapchain

# Swap Chain

---

- **Create Swapchain**

```
QueueFamilyIndices indices = findQueueFamilies(physicalDevice);
uint32_t queueFamilyIndices[] = { indices.graphicsFamily.value(), indices.presentFamily.value() };

if (indices.graphicsFamily != indices.presentFamily) {
    createInfo.imageSharingMode = VK_SHARING_MODE_CONCURRENT;
    createInfo.queueFamilyIndexCount = 2;
    createInfo.pQueueFamilyIndices = queueFamilyIndices;
}
else {
    createInfo.imageSharingMode = VK_SHARING_MODE_EXCLUSIVE;
}
```

- **VK\_SHARING\_MODE\_EXCLUSIVE**: An image is owned by one queue family at a time and ownership must be explicitly transferred before using it in another queue family. This option offers the best performance.
- **VK\_SHARING\_MODE\_CONCURRENT**: Images can be used across multiple queue families without explicit ownership transfers.

# Swap Chain

---

- **Create Swapchain**

```
createInfo.preTransform = swapChainSupport.capabilities.currentTransform;  
createInfo.compositeAlpha = VK_COMPOSITE_ALPHA_OPAQUE_BIT_KHR;  
createInfo.presentMode = presentMode;  
createInfo.clipped = VK_TRUE;  
  
createInfo.oldSwapchain = VK_NULL_HANDLE;
```

```
VkSwapchainKHR swapChain;
```

```
if (vkCreateSwapchainKHR(device, &createInfo, nullptr, &swapChain) != VK_SUCCESS)  
{  
    throw std::runtime_error("failed to create swap chain!");  
}
```

```
std::vector<VkImage> swapChainImages;
```

- Fetches the image handles currently available in the swap chain and stores them in the `swapChainImages` vector

```
vkGetSwapchainImagesKHR(device, swapChain, &imageCount, nullptr);  
swapChainImages.resize(imageCount);  
vkGetSwapchainImagesKHR(device, swapChain, &imageCount, swapChainImages.data());
```

# Image Views and Framebuffer

- **Image View** : Access to the image

```
std::vector<VkImageView> swapChainImageViews;
```

```
void createImageViews() {
    swapChainImageViews.resize(swapChainImages.size());

    for (size_t i = 0; i < swapChainImages.size(); i++) {

        VkImageViewCreateInfo createInfo{};
        createInfo.sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO;
        createInfo.image = swapChainImages[i];

        createInfo.viewType = VK_IMAGE_VIEW_TYPE_2D;
        createInfo.format = swapChainImageFormat;

        createInfo.components.r = VK_COMPONENT_SWIZZLE_IDENTITY;
        createInfo.components.g = VK_COMPONENT_SWIZZLE_IDENTITY;
        createInfo.components.b = VK_COMPONENT_SWIZZLE_IDENTITY;
        createInfo.components.a = VK_COMPONENT_SWIZZLE_IDENTITY;

        createInfo.subresourceRange.aspectMask = VK_IMAGE_ASPECT_COLOR_BIT;
        createInfo.subresourceRange.baseMipLevel = 0;
        createInfo.subresourceRange.levelCount = 1;
        createInfo.subresourceRange.baseArrayLayer = 0;
        createInfo.subresourceRange.layerCount = 1;
    }
}
```

- **VkImageViewCreateInfo** : Structure specifying parameters of a newly created image view
  - **sType** : **VkStructureType** value identifying this structure.
  - **pNext** : NULL or a pointer to a structure extending this structure.
  - **Flags** : A bitmask of **VkImageViewCreateFlagBits** specifying additional parameters of the image view.
  - **image** : **VkImage** on which the view will be created.
  - **viewType** : **VkImageViewType** value specifying the type of the image view.
  - **format** : **VkFormat** specifying the format and type used to interpret texel blocks of the image.
  - **components** : **VkComponentMapping** structure specifying a remapping of color components (or of depth or stencil components after they have been converted into color components).
  - **subresourceRange** : **VkImageSubresourceRange** structure selecting the set of mipmap levels and array layers to be accessible to the view.

# Image Views and Framebuffer

---

- **Image View** : Access to the image

```
if (vkCreateImageView(device, &createInfo, nullptr, &swapChainImageViews[i]) != VK_SUCCESS) {  
    throw std::runtime_error("failed to create image views!");  
}
```

# Shader modules

```
void createGraphicsPipeline() {  
    auto vertShaderCode = readFile("shaders/vert.spv");  
    auto fragShaderCode = readFile("shaders/frag.spv");  
  
    VkShaderModule vertShaderModule = createShaderModule(vertShaderCode);  
    VkShaderModule fragShaderModule = createShaderModule(fragShaderCode);  
  
    VkPipelineShaderStageCreateInfo vertShaderStageInfo{};  
    vertShaderStageInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;  
    vertShaderStageInfo.stage = VK_SHADER_STAGE_VERTEX_BIT;  
    vertShaderStageInfo.module = vertShaderModule;  
    vertShaderStageInfo.pName = "main";  
  
    VkPipelineShaderStageCreateInfo fragShaderStageInfo{};  
    fragShaderStageInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_SHADER_STAGE_CREATE_INFO;  
    fragShaderStageInfo.stage = VK_SHADER_STAGE_FRAGMENT_BIT;  
    fragShaderStageInfo.module = fragShaderModule;  
    fragShaderStageInfo.pName = "main";  
  
    VkPipelineShaderStageCreateInfo shaderStages[] = { vertShaderStageInfo, fragShaderStageInfo };  
  
    vkDestroyShaderModule(device, fragShaderModule, nullptr);  
    vkDestroyShaderModule(device, vertShaderModule, nullptr);  
}
```

- **VkShaderModule** : Opaque handle to a shader module object
- **VkPipelineShaderStageCreateInfo** : Structure specifying parameters of a newly created pipeline shader stage

# Shader modules

- Loading a shader and Creating shader modules

```
static std::vector<char> readFile(const std::string& filename) {
    std::ifstream file(filename, std::ios::ate | std::ios::binary);

    if (!file.is_open()) {
        throw std::runtime_error("failed to open file!");
    }
}
```

```
VkShaderModule createShaderModule(const std::vector<char>& code) {

    VkShaderModuleCreateInfo createInfo{};
    createInfo.sType = VK_STRUCTURE_TYPE_SHADER_MODULE_CREATE_INFO;
    createInfo.codeSize = code.size();
    createInfo.pCode = reinterpret_cast<const uint32_t*>(code.data());

    VkShaderModule shaderModule;
    if (vkCreateShaderModule(device, &createInfo, nullptr, &shaderModule) != VK_SUCCESS) {
        throw std::runtime_error("failed to create shader module!");
    }
}
```

- **vkCreateShaderModule Parameters** : logical device, pointer to create info structure, optional pointer to custom allocators and handle output variable

# Pipeline Fixed Functions

---

- **Vertex Input and Primitive Assembly**

```
VkPipelineVertexInputStateCreateInfo vertexInputInfo{};  
vertexInputInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_VERTEX_INPUT_STATE_CREATE_INFO;  
vertexInputInfo.vertexBindingDescriptionCount = 0;  
vertexInputInfo.vertexAttributeDescriptionCount = 0;
```

- **VkPipelineVertexInputStateCreateInfo** : Structure specifying parameters of a newly created pipeline vertex input state
  - **sType** : `VkStructureType` value identifying this structure.
  - **pNext** : NULL or a pointer to a structure extending this structure.
  - **flags** : reserved for future use.
  - **vertexBindingDescriptionCount** : The number of vertex binding descriptions provided in `pVertexBindingDescriptions`.
  - **pVertexBindingDescriptions** : A pointer to an array of `VkVertexInputBindingDescription` structures.
  - **vertexAttributeDescriptionCount** : The number of vertex attribute descriptions provided in `pVertexAttributeDescriptions`.
  - **pVertexAttributeDescriptions** : A pointer to an array of `VkVertexInputAttributeDescription` structures.



# Pipeline Fixed Functions

---

- **Vertex Input and Primitive Assembly**

```
VkPipelineInputAssemblyStateCreateInfo inputAssembly{};  
inputAssembly.sType = VK_STRUCTURE_TYPE_PIPELINE_INPUT_ASSEMBLY_STATE_CREATE_INFO;  
inputAssembly.topology = VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST;  
inputAssembly.primitiveRestartEnable = VK_FALSE;
```

- **VkPipelineInputAssemblyStateCreateInfo** : Structure specifying parameters of a newly created pipeline input assembly state
  - **sType** is a `VkStructureType` value identifying this structure.
  - **pNext** is NULL or a pointer to a structure extending this structure.
  - **flags** is reserved for future use.
  - **topology** is a `VkPrimitiveTopology` defining the primitive topology, as described below.
  - **primitiveRestartEnable** controls whether a special vertex index value is treated as restarting the assembly of primitives.
    - `VK_PRIMITIVE_TOPOLOGY_POINT_LIST`: points from vertices
    - `VK_PRIMITIVE_TOPOLOGY_LINE_LIST`: line from every 2 vertices without reuse
    - `VK_PRIMITIVE_TOPOLOGY_LINE_STRIP`: the end vertex of every line is used as start vertex for the next line
    - `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_LIST`: triangle from every 3 vertices without reuse
    - `VK_PRIMITIVE_TOPOLOGY_TRIANGLE_STRIP`: the second and third vertex of every triangle are used as first two vertices of the next triangle

# Pipeline Fixed Functions

- **Viewport and Scissors**

```
VkViewport viewport{};
viewport.x = 0.0f;
viewport.y = 0.0f;
viewport.width = (float)swapChainExtent.width;
viewport.height = (float)swapChainExtent.height;
viewport.minDepth = 0.0f;
viewport.maxDepth = 1.0f;
```

```
VkRect2D scissor{};
scissor.offset = { 0, 0 };
scissor.extent = swapChainExtent;
```

```
VkPipelineViewportStateCreateInfo viewportState{};
viewportState.sType = VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO;
viewportState.viewportCount = 1;
viewportState.scissorCount = 1;
```

```
VkPipelineViewportStateCreateInfo viewportState{};
viewportState.sType = VK_STRUCTURE_TYPE_PIPELINE_VIEWPORT_STATE_CREATE_INFO;
viewportState.viewportCount = 1;
viewportState.pViewports = &viewport;
viewportState.scissorCount = 1;
viewportState.pScissors = &scissor;
```

- **Viewports** : Define the transformation from the image to the framebuffer
- **Scissor rectangles** : Define in which regions pixels will actually be stored
- **VkPipelineViewportStateCreateInfo** : Structure specifying parameters of a newly created pipeline viewport state
  - **pViewports** : A pointer to an array of VkViewport structures, defining the viewport transforms.
  - **scissorCount** : The number of scissors and must match the number of viewports.
  - **pScissors** : A pointer to an array of VkRect2D structures defining the rectangular bounds of the scissor for the corresponding viewport.

# Pipeline Fixed Functions

- **Rasterizer**

```
VkPipelineRasterizationStateCreateInfo rasterizer{};  
rasterizer.sType = VK_STRUCTURE_TYPE_PIPELINE_RASTERIZATION_STATE_CREATE_INFO;  
rasterizer.depthClampEnable = VK_FALSE;  
rasterizer.rasterizerDiscardEnable = VK_FALSE;  
rasterizer.polygonMode = VK_POLYGON_MODE_FILL;  
rasterizer.lineWidth = 1.0f;  
rasterizer.cullMode = VK_CULL_MODE_BACK_BIT;  
rasterizer.frontFace = VK_FRONT_FACE_CLOCKWISE;  
rasterizer.depthBiasEnable = VK_FALSE;
```

- `VK_POLYGON_MODE_FILL`: fill the area of the polygon with fragments
- `VK_POLYGON_MODE_LINE`: polygon edges are drawn as lines
- `VK_POLYGON_MODE_POINT`: polygon vertices are drawn as points

- **VkPipelineRasterizationStateCreateInfo** - Structure specifying parameters of a newly created pipeline rasterization state
  - **depthClampEnable** controls whether to clamp the fragment's depth values as described in Depth Test.
  - **rasterizerDiscardEnable** controls whether primitives are discarded immediately before the rasterization stage.
  - **polygonMode** is the triangle rendering mode. See `VkPolygonMode`.
  - **cullMode** is the triangle facing direction used for primitive culling. See `VkCullModeFlagBits`.
  - **frontFace** is a `VkFrontFace` value specifying the front-facing triangle orientation to be used for culling.
  - **depthBiasEnable** controls whether to bias fragment depth values.

# Pipeline Fixed Functions

---

- **Anti-aliasing**

```
VkPipelineMultisampleStateCreateInfo multisampling{};
multisampling.sType = VK_STRUCTURE_TYPE_PIPELINE_MULTISAMPLE_STATE_CREATE_INFO;
multisampling.sampleShadingEnable = VK_FALSE;
multisampling.rasterizationSamples = VK_SAMPLE_COUNT_1_BIT;
multisampling.minSampleShading = 1.0f; // Optional
multisampling.pSampleMask = nullptr; // Optional
multisampling.alphaToCoverageEnable = VK_FALSE; // Optional
multisampling.alphaToOneEnable = VK_FALSE; // Optional
```

- **VkPipelineMultisampleStateCreateInfo** : Structure specifying parameters of a newly created pipeline multisample state
  - **rasterizationSamples** is a VkSampleCountFlagBits value specifying the number of samples used in rasterization.
  - **sampleShadingEnable** can be used to enable Sample\_Shading.
  - **minSampleShading** specifies a minimum fraction of sample shading if sampleShadingEnable is set to VK\_TRUE.
  - **pSampleMask** is a pointer to an array of VkSampleMask values used in the sample mask test.
  - **alphaToCoverageEnable** controls whether a temporary coverage value is generated based on the alpha component of the fragment's first color output as specified in the Multisample Coverage section.
  - **alphaToOneEnable** controls whether the alpha component of the fragment's first color output is replaced with one as described in Multisample Coverage.
- **Depth and Stencil Buffer**
  - **VkPipelineDepthStencilStateCreateInfo** : Structure specifying parameters of a newly created pipeline depth stencil state

# Pipeline Fixed Functions

- **Color Blending**

```
VkPipelineColorBlendAttachmentState colorBlendAttachment{};
    colorBlendAttachment.colorWriteMask = VK_COLOR_COMPONENT_R_BIT | VK_COLOR_COMPONENT_G_BIT | VK_COLOR_COMPONENT_B_BIT |
VK_COLOR_COMPONENT_A_BIT;
    colorBlendAttachment.blendEnable = VK_FALSE;
    colorBlendAttachment.srcColorBlendFactor = VK_BLEND_FACTOR_ONE; // Optional
    colorBlendAttachment.dstColorBlendFactor = VK_BLEND_FACTOR_ZERO; // Optional
    colorBlendAttachment.colorBlendOp = VK_BLEND_OP_ADD; // Optional
    colorBlendAttachment.srcAlphaBlendFactor = VK_BLEND_FACTOR_ONE; // Optional
    colorBlendAttachment.dstAlphaBlendFactor = VK_BLEND_FACTOR_ZERO; // Optional
    colorBlendAttachment.alphaBlendOp = VK_BLEND_OP_ADD; // Optional
```

- **VkPipelineColorBlendAttachmentState** : Structure specifying a pipeline color blend attachment state
  - **blendEnable** controls whether blending is enabled for the corresponding color attachment. If blending is not enabled, the source fragment's color for that attachment is passed through unmodified.
  - **srcColorBlendFactor** selects which blend factor is used to determine the source factors ( $S_r, S_g, S_b$ ).
  - **dstColorBlendFactor** selects which blend factor is used to determine the destination factors ( $D_r, D_g, D_b$ ).
  - **colorBlendOp** selects which blend operation is used to calculate the RGB values to write to the color attachment.
  - **srcAlphaBlendFactor** selects which blend factor is used to determine the source factor  $S_a$ .
  - **dstAlphaBlendFactor** selects which blend factor is used to determine the destination factor  $D_a$ .
  - **alphaBlendOp** selects which blend operation is used to calculate the alpha values to write to the color attachment.
  - **colorWriteMask** : A bitmask of VkColorComponentFlagBits specifying which of the R, G, B, and/or A components are enabled for writing, as described for the Color Write Mask.

# Pipeline Fixed Functions

- **Color Blending**

```
VkPipelineColorBlendStateCreateInfo colorBlending{};
colorBlending.sType = VK_STRUCTURE_TYPE_PIPELINE_COLOR_BLEND_STATE_CREATE_INFO;
colorBlending.logicOpEnable = VK_FALSE;
colorBlending.logicOp = VK_LOGIC_OP_COPY;
colorBlending.attachmentCount = 1;
colorBlending.pAttachments = &colorBlendAttachment;
colorBlending.blendConstants[0] = 0.0f;
colorBlending.blendConstants[1] = 0.0f;
colorBlending.blendConstants[2] = 0.0f;
colorBlending.blendConstants[3] = 0.0f;
```

- **VkPipelineColorBlendStateCreateInfo** : Structure specifying parameters of a newly created pipeline color blend state
  - **logicOpEnable** controls whether to apply Logical Operations.
  - **logicOp** selects which logical operation to apply.
  - **attachmentCount** : The number of VkPipelineColorBlendAttachmentState elements in pAttachments.
  - **pAttachments** : A pointer to an array of VkPipelineColorBlendAttachmentState structures defining blend state for each color attachment.
  - **blendConstants** : A pointer to an array of four values used as the R, G, B, and A components of the blend constant that are used in blending, depending on the blend factor.

# Pipeline Fixed Functions

---

- Shader Uniform Value

```
VkPipelineLayout pipelineLayout;
```

```
VkPipelineLayoutCreateInfo pipelineLayoutInfo{};  
    pipelineLayoutInfo.sType = VK_STRUCTURE_TYPE_PIPELINE_LAYOUT_CREATE_INFO;  
    pipelineLayoutInfo.setLayoutCount = 0; // Optional  
    pipelineLayoutInfo.pSetLayouts = nullptr; // Optional  
    pipelineLayoutInfo.pushConstantRangeCount = 0; // Optional  
    pipelineLayoutInfo.pPushConstantRanges = nullptr; // Optional  
  
    if (vkCreatePipelineLayout(device, &pipelineLayoutInfo, nullptr, &pipelineLayout) != VK_SUCCESS) {  
        throw std::runtime_error("failed to create pipeline layout!");  
    }
```

# Pipeline Fixed Functions

---

- Create the graphics pipeline

```
VkGraphicsPipelineCreateInfo pipelineInfo{};
pipelineInfo.sType = VK_STRUCTURE_TYPE_GRAPHICS_PIPELINE_CREATE_INFO;
pipelineInfo.stageCount = 2;
pipelineInfo.pStages = shaderStages;
pipelineInfo.pVertexInputState = &vertexInputInfo;
pipelineInfo.pInputAssemblyState = &inputAssembly;
pipelineInfo.pViewportState = &viewportState;
pipelineInfo.pRasterizationState = &rasterizer;
pipelineInfo.pMultisampleState = &multisampling;
pipelineInfo.pColorBlendState = &colorBlending;
pipelineInfo.pDynamicState = &dynamicState;
pipelineInfo.layout = pipelineLayout;
pipelineInfo.renderPass = renderPass;
pipelineInfo.subpass = 0;
pipelineInfo.basePipelineHandle = VK_NULL_HANDLE;
```

- **VkGraphicsPipelineCreateInfo** : Structure specifying parameters of a newly created graphics pipeline



# Pipeline Fixed Functions

---

- **VkGraphicsPipelineCreateInfo** : Structure specifying parameters of a newly created graphics pipeline
  - **stageCount** : The number of entries in the pStages array.
  - **pStages** : A pointer to an array of stageCount VkPipelineShaderStageCreateInfo structures describing the set of the shader stages to be included in the graphics pipeline.
  - **pVertexInputState** : A pointer to a VkPipelineVertexInputStateCreateInfo structure.
  - **pInputAssemblyState** : A pointer to a VkPipelineInputAssemblyStateCreateInfo structure which determines input assembly behavior for vertex shading, as described in Drawing Commands.
  - **pTessellationState** : A pointer to a VkPipelineTessellationStateCreateInfo structure defining tessellation state used by tessellation shaders.
  - **pViewportState** : A pointer to a VkPipelineViewportStateCreateInfo structure defining viewport state used when rasterization is enabled.
  - **pRasterizationState** : A pointer to a VkPipelineRasterizationStateCreateInfo structure defining rasterization state.
  - **pMultisampleState** : A pointer to a VkPipelineMultisampleStateCreateInfo structure defining multisample state used when rasterization is enabled.
  - **pDepthStencilState** : A pointer to a VkPipelineDepthStencilStateCreateInfo structure defining depth/stencil state used when rasterization is enabled for depth or stencil attachments accessed during rendering.
  - **pColorBlendState** : A pointer to a VkPipelineColorBlendStateCreateInfo structure defining color blend state used when rasterization is enabled for any color attachments accessed during rendering.
  - **pDynamicState** : A pointer to a VkPipelineDynamicStateCreateInfo structure defining which properties of the pipeline state object are dynamic and can be changed independently of the pipeline state.
  - layout is the description of binding locations used by both the pipeline and descriptor sets used with the pipeline.
  - **renderPass** : A handle to a render pass object describing the environment in which the pipeline will be used.
  - **subpass** : The index of the subpass in the render pass where this pipeline will be used.
  - **basePipelineHandle** : A pipeline to derive from.
  - **basePipelineIndex** : An index into the pCreateInfos parameter to use as a pipeline to derive from.

# Pipeline Fixed Functions

---

- Create the graphics pipeline

```
VkPipeline graphicsPipeline;
```

```
if (vkCreateGraphicsPipelines(device, VK_NULL_HANDLE, 1, &pipelineInfo, nullptr, &graphicsPipeline) != VK_SUCCESS) {  
    throw std::runtime_error("failed to create graphics pipeline!");  
}
```

# Render pass

```
void createRenderPass() {  
    VkAttachmentDescription colorAttachment{};  
    colorAttachment.format = swapChainImageFormat;  
    colorAttachment.samples = VK_SAMPLE_COUNT_1_BIT;  
    colorAttachment.loadOp = VK_ATTACHMENT_LOAD_OP_CLEAR;  
    colorAttachment.storeOp = VK_ATTACHMENT_STORE_OP_STORE;  
    colorAttachment.stencilLoadOp = VK_ATTACHMENT_LOAD_OP_DONT_CARE;  
    colorAttachment.stencilStoreOp = VK_ATTACHMENT_STORE_OP_DONT_CARE;  
    colorAttachment.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;  
    colorAttachment.finalLayout = VK_IMAGE_LAYOUT_PRESENT_SRC_KHR;  
  
    VkAttachmentReference colorAttachmentRef{};  
    colorAttachmentRef.attachment = 0;  
    colorAttachmentRef.layout = VK_IMAGE_LAYOUT_COLOR_ATTACHMENT_OPTIMAL;  
  
    VkSubpassDescription subpass{};  
    subpass.pipelineBindPoint = VK_PIPELINE_BIND_POINT_GRAPHICS;  
    subpass.colorAttachmentCount = 1;  
    subpass.pColorAttachments = &colorAttachmentRef;  
  
    VkRenderPassCreateInfo renderPassInfo{};  
    renderPassInfo.sType = VK_STRUCTURE_TYPE_RENDER_PASS_CREATE_INFO;  
    renderPassInfo.attachmentCount = 1;  
    renderPassInfo.pAttachments = &colorAttachment;  
    renderPassInfo.subpassCount = 1;  
    renderPassInfo.pSubpasses = &subpass;  
  
    if (vkCreateRenderPass(device, &renderPassInfo, nullptr, &renderPass) != VK_SUCCESS) {  
        throw std::runtime_error("failed to create render pass!");  
    }  
}
```

- **VkAttachmentDescription** : Structure specifying an attachment description
- **VkAttachmentReference** : Structure specifying an attachment reference
- **VkSubpassDescription** : Structure specifying a subpass description
- **VkRenderPassCreateInfo** : Structure specifying parameters of a newly created render pass

# Vertex and Fragment Shader

---

- Vertex Shader

```
#version 450
```

```
layout(location = 0) out vec3 fragColor;
```

```
vec2 positions[3] = vec2[](
    vec2(0.0, -0.5),
    vec2(0.5, 0.5),
    vec2(-0.5, 0.5)
);
```

```
vec3 colors[3] = vec3[](
    vec3(1.0, 0.0, 0.0),
    vec3(0.0, 1.0, 0.0),
    vec3(0.0, 0.0, 1.0)
);
```

```
void main() {
    gl_Position = vec4(positions[gl_VertexIndex], 0.0, 1.0);
    fragColor = colors[gl_VertexIndex];
}
```

- Fragment Shader

```
#version 450
```

```
layout(location = 0) in vec3 fragColor;
```

```
layout(location = 0) out vec4 outColor;
```

```
void main() {
    outColor = vec4(fragColor, 1.0);
}
```

# Shader Compile

---

- `C:/VulkanSDK/x.x.x.x/Bin32/glslc.exe shader.vert -o vert.spv`
- `C:/VulkanSDK/x.x.x.x/Bin32/glslc.exe shader.frag -o frag.spv`

# Frame Buffers

```
void createFramebuffers() {
    swapChainFramebuffers.resize(swapChainImageViews.size());

    for (size_t i = 0; i < swapChainImageViews.size(); i++) {
        VkImageView attachments[] = {
            swapChainImageViews[i]
        };

        VkFramebufferCreateInfo framebufferInfo{};
        framebufferInfo.sType =
VK_STRUCTURE_TYPE_FRAMEBUFFER_CREATE_INFO;
        framebufferInfo.renderPass = renderPass;
        framebufferInfo.attachmentCount = 1;
        framebufferInfo.pAttachments = attachments;
        framebufferInfo.width = swapChainExtent.width;
        framebufferInfo.height = swapChainExtent.height;
        framebufferInfo.layers = 1;

        if (vkCreateFramebuffer(device, &framebufferInfo, nullptr,
&swapChainFramebuffers[i]) != VK_SUCCESS) {
            throw std::runtime_error("failed to create framebuffer!");
        }
    }
}
```

- **VkFramebufferCreateInfo** - Structure specifying parameters of a newly created framebuffer
  - **renderPass** is a render pass defining what render passes the framebuffer will be compatible with.
  - **attachmentCount** is the number of attachments.
  - **pAttachments** is a pointer to an array of VkImageView handles, each of which will be used as the corresponding attachment in a render pass instance.
  - **width**, **height** and **layers** define the dimensions of the framebuffer.

# Command Buffers

```
void createCommandBuffer() {
    VkCommandBufferAllocateInfo allocInfo{};
    allocInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_ALLOCATE_INFO;
    allocInfo.commandPool = commandPool;
    allocInfo.level = VK_COMMAND_BUFFER_LEVEL_PRIMARY;
    allocInfo.commandBufferCount = 1;

    if (vkAllocateCommandBuffers(device, &allocInfo, &commandBuffer) != VK_SUCCESS) {
        throw std::runtime_error("failed to allocate command buffers!");
    }
}

void recordCommandBuffer(VkCommandBuffer commandBuffer, uint32_t imageIndex) {
    VkCommandBufferBeginInfo beginInfo{};
    beginInfo.sType = VK_STRUCTURE_TYPE_COMMAND_BUFFER_BEGIN_INFO;

    if (vkBeginCommandBuffer(commandBuffer, &beginInfo) != VK_SUCCESS) {
        throw std::runtime_error("failed to begin recording command buffer!");
    }

    VkRenderPassBeginInfo renderPassInfo{};
    renderPassInfo.sType = VK_STRUCTURE_TYPE_RENDER_PASS_BEGIN_INFO;
    renderPassInfo.renderPass = renderPass;
    renderPassInfo.framebuffer = swapChainFramebuffers[imageIndex];
    renderPassInfo.renderArea.offset = { 0, 0 };
    renderPassInfo.renderArea.extent = swapChainExtent;

    VkClearColorValue clearColor = { {0.0f, 0.0f, 0.0f, 1.0f} };
    renderPassInfo.clearValueCount = 1;
    renderPassInfo.pClearValues = &clearColor;
```

```
    vkCmdBeginRenderPass(commandBuffer, &renderPassInfo,
                          VK_SUBPASS_CONTENTS_INLINE);

    vkCmdBindPipeline(commandBuffer, VK_PIPELINE_BIND_POINT_GRAPHICS,
                      graphicsPipeline);

    VkViewport viewport{};
    viewport.x = 0.0f;
    viewport.y = 0.0f;
    viewport.width = (float)swapChainExtent.width;
    viewport.height = (float)swapChainExtent.height;
    viewport.minDepth = 0.0f;
    viewport.maxDepth = 1.0f;
    vkCmdSetViewport(commandBuffer, 0, 1, &viewport);

    VkRect2D scissor{};
    scissor.offset = { 0, 0 };
    scissor.extent = swapChainExtent;
    vkCmdSetScissor(commandBuffer, 0, 1, &scissor);

    vkCmdDraw(commandBuffer, 3, 1, 0, 0);

    vkCmdEndRenderPass(commandBuffer);

    if (vkEndCommandBuffer(commandBuffer) != VK_SUCCESS) {
        throw std::runtime_error("failed to record command buffer!");
    }
}
```

# Command Buffers

---

- **VkCommandPool** : Opaque handle to a command pool object
- **VkCommandPoolCreateInfo** : Structure specifying parameters of a newly created command pool
- **VkCommandBuffer** : Opaque handle to a command buffer object
- **VkCommandBufferAllocateInfo** : Structure specifying the allocation parameters for command buffer object
- **VkCommandBufferBeginInfo** : Structure specifying a command buffer begin operation
- **vkAllocateCommandBuffers** - Allocate command buffers from an existing command pool
- **vkBeginCommandBuffer** : Start recording a command buffer
- **VkRenderPassBeginInfo** : Structure specifying render pass begin information
- **vkCmdBeginRenderPass** : Begin a new render pass
- **vkCmdBindPipeline** : Bind a pipeline object to a command buffer
- **vkCmdEndRenderPass** : End the current render pass
- **vkCmdDraw** : Draw primitives



# Rendering and Presenting a Frame

```
void drawFrame() {
    vkWaitForFences(device, 1, &inFlightFence, VK_TRUE, UINT64_MAX);
    vkResetFences(device, 1, &inFlightFence);

    uint32_t imageIndex;
    vkAcquireNextImageKHR(device, swapChain, UINT64_MAX,
        imageAvailableSemaphore, VK_NULL_HANDLE, &imageIndex);

    vkResetCommandBuffer(commandBuffer, /*VkCommandBufferResetFlagBits*/ 0);
    recordCommandBuffer(commandBuffer, imageIndex);

    VkSubmitInfo submitInfo{};
    submitInfo.sType = VK_STRUCTURE_TYPE_SUBMIT_INFO;

    VkSemaphore waitSemaphores[] = { imageAvailableSemaphore };
    VkPipelineStageFlags waitStages[] =
        { VK_PIPELINE_STAGE_COLOR_ATTACHMENT_OUTPUT_BIT };
    submitInfo.waitSemaphoreCount = 1;
    submitInfo.pWaitSemaphores = waitSemaphores;
    submitInfo.pWaitDstStageMask = waitStages;

    submitInfo.commandBufferCount = 1;
    submitInfo.pCommandBuffers = &commandBuffer;

    VkSemaphore signalSemaphores[] = { renderFinishedSemaphore };
    submitInfo.signalSemaphoreCount = 1;
    submitInfo.pSignalSemaphores = signalSemaphores;
```

```
    if (vkQueueSubmit(graphicsQueue, 1, &submitInfo, inFlightFence) !=
        VK_SUCCESS) {
        throw std::runtime_error("failed to submit draw command buffer!");
    }

    VkPresentInfoKHR presentInfo{};
    presentInfo.sType = VK_STRUCTURE_TYPE_PRESENT_INFO_KHR;

    presentInfo.waitSemaphoreCount = 1;
    presentInfo.pWaitSemaphores = signalSemaphores;

    VkSwapchainKHR swapChains[] = { swapChain };
    presentInfo.swapchainCount = 1;
    presentInfo.pSwapchains = swapChains;

    presentInfo.pImageIndices = &imageIndex;

    vkQueuePresentKHR(presentQueue, &presentInfo);
}
```

# Rendering and Presenting a Frame

---

- **vkWaitForFences** : Wait for one or more fences to become signaled
- **vkResetFences** : Resets one or more fence objects
- **vkAcquireNextImageKHR** : Retrieve the index of the next available presentable image
- **vkResetCommandBuffer** : Reset a command buffer to the initial state
- **VkSubmitInfo** : Structure specifying a queue submit operation
- **VkSemaphore** : Opaque handle to a semaphore object
- **VkPipelineStageFlags** : Bitmask of VkPipelineStageFlagBits
- **VkPresentInfoKHR** : Structure describing parameters of a queue presentation
- **VkSwapchainKHR** : Opaque handle to a swapchain object
- **vkQueuePresentKHR** : Queue an image for presentation

# Setting Up Synchronization Mechanisms

---

```
void createSyncObjects() {  
    VkSemaphoreCreateInfo semaphoreInfo{};  
    semaphoreInfo.sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO;  
  
    VkFenceCreateInfo fenceInfo{};  
    fenceInfo.sType = VK_STRUCTURE_TYPE_FENCE_CREATE_INFO;  
    fenceInfo.flags = VK_FENCE_CREATE_SIGNALED_BIT;  
  
    if (vkCreateSemaphore(device, &semaphoreInfo, nullptr, &imageAvailableSemaphore) != VK_SUCCESS ||  
        vkCreateSemaphore(device, &semaphoreInfo, nullptr, &renderFinishedSemaphore) != VK_SUCCESS ||  
        vkCreateFence(device, &fenceInfo, nullptr, &inFlightFence) != VK_SUCCESS) {  
        throw std::runtime_error("failed to create synchronization objects for a frame!");  
    }  
}
```

- **VkSemaphoreCreateInfo** : Structure specifying parameters of a newly created semaphore
- **VkFenceCreateInfo** : Structure specifying parameters of a newly created fence

# To be continued

---

## Managing Resources in Vulkan (Buffers)

- Vertex Buffers
- Uniform Buffers